

王道考研——数据结构

WWW.CSKAOYAN.COM

第二章 线性表

1

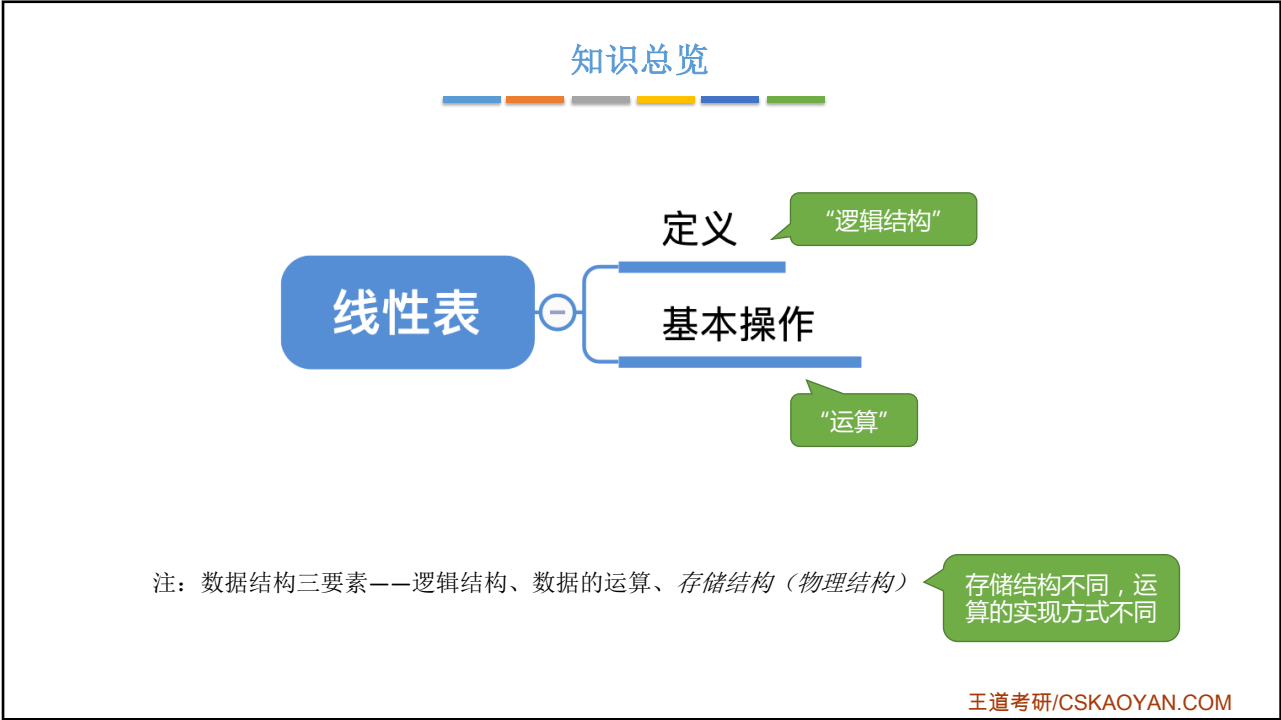
本节内容

线性表

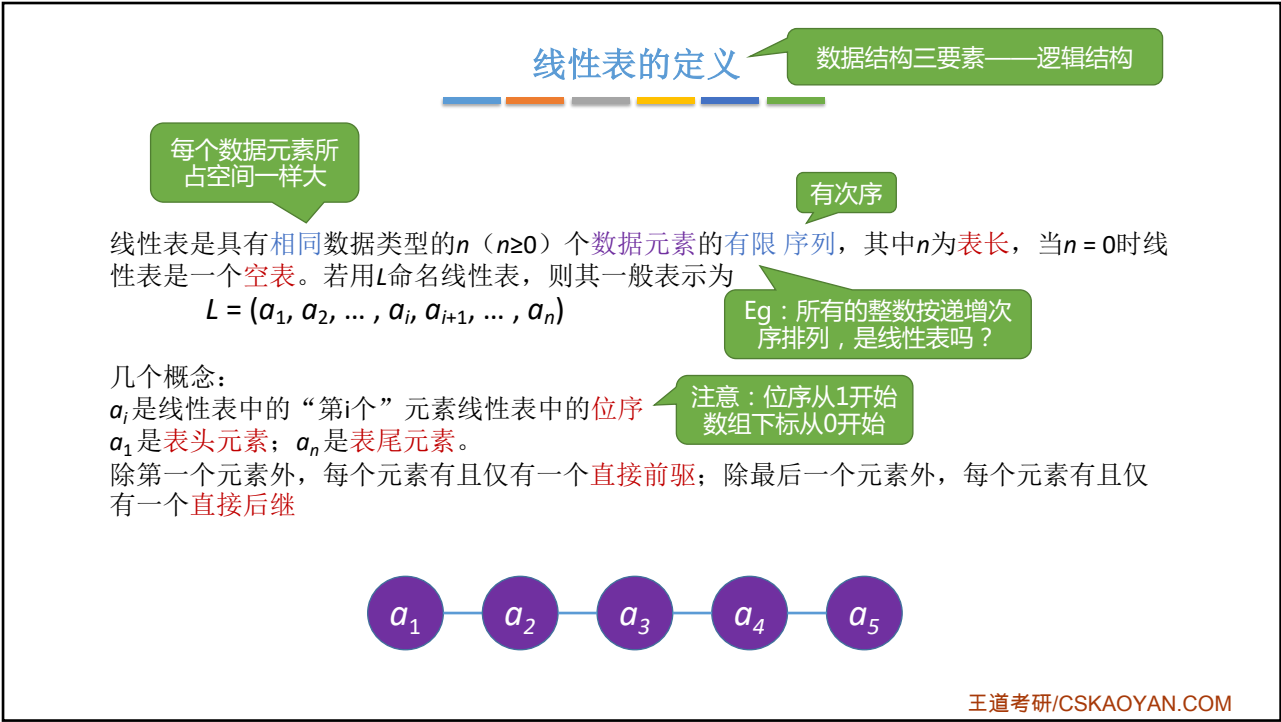
定义、
基本操作

王道考研/CSKAOYAN.COM

2



3



4

线性表的定义

线性表 —— Linear List



吃惊

linear

英 ['liniə(r)] 美 ['liniər]

adj. 线的, 线型的; 直线的, 线状的; 长度的

词根: Line 线
Eg: Sky line baby

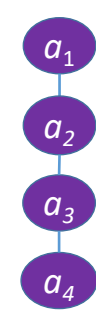
list

英 [list] 美 [lis]

n. **列表**, 清单, 目录;

Eg: a list of ...
一连串、一系列的...

	A	B	C	D
1	学号	姓名	性别	专业
2	1120112100	张三	男	挖掘机
3	1120112101	李四	女	挖掘机
4	1120112102	王五	男	数据挖掘
5	1120112103	赵六	男	挖掘机
6	1120112104	钱七	女	挖掘机
7	1120112105	狗剩	男	数据挖掘
8	1120112106	铁柱	男	数据挖掘
9	1120112107	如花	女	数据挖掘
10	1120112108	二狗	男	数据挖掘
11	1120112109	傻根儿	男	数据挖掘
12	1120112110	旺财	女	数据挖掘





王道考研/CSKAOYAN.COM

5

线性表的基本操作

WHY?



咸鱼要翻身

为什么要实现对数据结构的基本操作?

①团队合作编程, 你定义的数据结构要让别人能够很方便的使用(封装)
②将常用的操作/运算封装成函数, 避免重复工作, 降低出错风险

Tips: 比起学会 "How", 更重要的是想明白 "Why"

数据结构三要素——“运算”

王道考研/CSKAOYAN.COM

6

线性表的基本操作

InitList(&L): **初始化**表。构造一个空的线性表L, **分配内存空间**。

DestroyList(&L): **销毁**操作。销毁线性表, 并**释放**线性表L所占用的**内存空间**。

从无到有
从有到无

ListInsert(&L,i,e): **插入**操作。在表L中的第i个位置上插入指定元素e。

ListDelete(&L,i,&e): **删除**操作。删除表L中第i个位置的元素, 并用e返回删除元素的值。

增、删

LocateElem(L,e): **按值查找**操作。在表L中查找具有给定关键字值的元素。

GetElem(L,i): **按位查找**操作。获取表L中第i个位置的元素的值。

改、查（“改”之前也要“查”）

其他常用操作:

Length(L): 求表长。返回线性表L的长度, 即L中数据元素的个数。

PrintList(L): 输出操作。按前后顺序输出线性表L的所有元素值。

Empty(L): 判空操作。若L为空表, 则返回true, 否则返回false。

Tips:

①对数据的操作（记忆思路）—— 创销、增删改查

②C语言函数的定义 —— <返回值类型> 函数名 (<参数1类型> 参数1, <参数2类型> 参数2,)

③实际开发中, 可根据实际需求定义其他的基本操作

④函数名和参数的形式、命名都可改变（Reference: 严蔚敏版《数据结构》）

⑤什么时候要传入引用“&”—— 对参数的修改结果需要“带回来”

为什么这里没有说明
各个参数的具体类型?

Key: 命名要有可读性

王道考研/CSKAOYAN.COM

7

线性表的基本操作

什么时候要传入参数的引用“&”—— 对参数的修改结果需要“带回来”

菜鸟工具 WEB 在线编辑器 SVG 在线编辑器 实例归档 菜鸟教程 输入关键字.....

点击运行 标准输入(stdin) C++ 在线工具 清空

```

1 #include<stdio.h>
2
3 void test(int x) {
4     x=1024;
5     printf("test函数内部 x=%d\n",x);
6 }
7
8 int main() {
9     int x = 1;
10    printf("调用test前 x=%d\n",x);
11    test(x);
12    printf("调用test后 x=%d\n",x);
13 }
    
```

对参数的修改
“没带回来”

调用test前 x=1
test函数内部 x=1024
调用test后 x=1

复制

内存

百度“C语言在线工具”

王道考研/CSKAOYAN.COM

8

线性表的基本操作

什么时候要传入参数的引用“&”——对参数的修改结果需要“带回来”

菜鸟工具 WEB 在线编辑器 SVG 在线编辑器 实例归档 菜鸟教程 输入关键字.....

点击运行 标准输入(stdin) C++ 在线工具 清空

```

1 #include<stdio.h>
2
3 void test(int & x) {
4     x=1024;
5     printf("test函数内部 x=%d\n",x);
6 }
7
8 int main() {
9     int x = 1;
10    printf("调用test前 x=%d\n",x);
11    test(x);
12    printf("调用test后 x=%d\n",x);
13 }

```

调用test前 x=1
test函数内部 x=1024
调用test后 x=1024

对参数的修改“带回来”了

内存

王道考研/CSKAOYAN.COM

9

线性表的基本操作

InitList(&L): 初始化表。构造一个空的线性表L，分配内存空间。

DestroyList(&L): 销毁操作。销毁线性表，并释放线性表L所占用的内存空间。

从无到有
从有到无

ListInsert(&L,i,e): 插入操作。在表L中的第i个位置上插入指定元素e。

ListDelete(&L,i,&e): 删除操作。删除表L中第i个位置的元素，并用e返回删除元素的值。

增、删

LocateElem(L,e): 按值查找操作。在表L中查找具有给定关键字值的元素。

GetElem(L,i): 按位查找操作。获取表L中第i个位置的元素的值。

改、查（本质是“定位”）

其他常用操作：

Length(L): 求表长。返回线性表L的长度，即L中数据元素的个数。

PrintList(L): 输出操作。按前后顺序输出线性表L的所有元素值。

Empty(L): 判空操作。若L为空表，则返回true，否则返回false。

Tips:

①对数据的操作（分析思路）—— 创销、增删改查

②C语言函数的定义—— <返回值类型> 函数名(<参数1类型> 参数1, <参数2类型> 参数2,)

③实际开发中，可根据实际需求定义其他的基本操作

④函数名和参数的形式、命名都可改变（Reference: 严蔚敏版《数据结构》）

⑤什么时候要传入参数的引用“&”——对参数的修改结果需要“带回来”

为什么此处没有说明
各个参数的具体类型？

Key: 命名要有可读性

王道考研/CSKAOYAN.COM

10

知识回顾与重要考点



王道考研/CSKAOYAN.COM

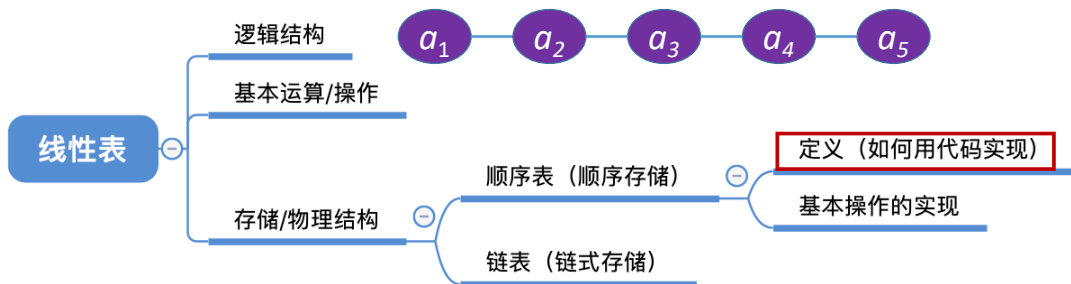
本节内容

顺序表 定义

王道考研/CSKAOYAN.COM

1

知识总览



王道考研/CSKAOYAN.COM

2

顺序表的定义

线性表 L 逻辑结构

线性表是具有**相同**数据类型的 n ($n \geq 0$) 个**数据元素**的有限序列

每个数据元素所占空间一样大

顺序表——用**顺序存储**的方式实现线性表顺序存储。把**逻辑上相邻**的元素存储在**物理位置上也相邻**的存储单元中，元素之间的关系由存储单元的邻接关系来体现。

```
typedef struct {
    int num;           //号数
    int people;        //人数
} Customer;
```

内存

LOC (L)

LOC (L)+数据元素的大小

LOC (L)+2*数据元素的大小

...

ElemType 就是你的顺序表中存放的数据元素类型

如何知道一个数据元素大小?
C语言 **sizeof(ElemType)**
Eg:
sizeof(int) = 4B
sizeof(Customer) = 8B

王道考研/CSKAOYAN.COM

3

顺序表的实现——静态分配

```
#define MaxSize 10
typedef struct{
    ElemType data[MaxSize];
    int length;
}SqList;
```

Sq: sequence —— 顺序, 序列

//定义最大长度

//用静态的“数组”存放数据元素

//顺序表的当前长度

//顺序表的类型定义（静态分配方式）

给各个数据元素分配连续的存储空间，大小为 $\text{MaxSize} * \text{sizeof}(\text{ElemType})$

王道考研/CSKAOYAN.COM

4

```

#include <stdio.h>
#define MaxSize 10 //定义最大长度
typedef struct{
    int data[MaxSize]; //用静态的“数组”存放数据元素
    int length; //顺序表的当前长度
}SqList; //顺序表的类型定义

//基本操作——初始化一个顺序表
void InitList(SqList &L){
    ② for(int i=0; i<MaxSize; i++){
        L.data[i]=0; //将所有数据元素设置为默认初始值
    }
    ③ L.length=0; //顺序表初始长度为0
}

int main() {
    ① SqList L; //声明一个顺序表
    InitList(L); //初始化顺序表
    //....未完待续, 后续操作
    return 0;
}

```

内存

②把各个数据元素的值设为默认值 (可省略)

①在内存中分配存储顺序表L的空间。包括: MaxSize*sizeof(ElemType) 和 存储 length 的空间

③将 Length 的值设为0

王道考研/CSKAOYAN.COM

5

```

/*不初始化数据元素, 内存不刷0*/
#include <stdio.h>
#define MaxSize 10 //定义最大长度
typedef struct{
    int data[MaxSize]; //用静态的“数组”存放数据元素
    int length; //顺序表的当前长度
}SqList; //顺序表的类型定义

//基本操作——初始化一个顺序表
void InitList(SqList &L){
    ③ L.length=0; //顺序表初始长度为0
}

int main() {
    ① SqList L; //声明一个顺序表
    InitList(L); //初始化顺序表
    //尝试“违规”打印整个 data 数组
    for(int i=0; i<MaxSize; i++){
        printf("data[%d]=%d\n", i, L.data[i]);
    }
    return 0;
}

```

内存

②把各个数据元素的值设为默认值 (可省略)

①在内存中分配存储顺序表L的空间。包括: MaxSize*sizeof(ElemType) 和 存储 length 的空间

③将 Length 的值设为0

思考: 这一步是否可省略?

王道考研/CSKAOYAN.COM

6

顺序表的实现——静态分配

```
#define MaxSize 10           //定义最大长度
typedef struct{
    ElemType data[MaxSize]; //用静态的“数组”存放数据元素
    int length;             //顺序表的当前长度
}SqList;                   //顺序表的类型定义
```



给各个数据元素分配连续的存储空间，大小为 $\text{MaxSize} * \text{sizeof}(\text{ElemType})$



Q: 如果“数组”存满了怎么办?

A: 可以放弃治疗，顺序表的表长刚开始确定后就无法更改（存储空间是静态的）

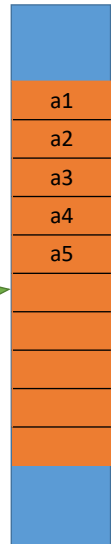
思考：如果刚开始就声明一个很大的内存空间呢？存在什么问题？



同学，浪费是不行的

王道考研/CSKAOYAN.COM

内存



7

顺序表的实现——动态分配

```
#define InitSize 10          //顺序表的初始长度
typedef struct{
    ElemType *data;          //指示动态分配数组的指针
    int MaxSize;             //顺序表的最大容量
    int length;             //顺序表的当前长度
} SeqList;                 //顺序表的类型定义（动态分配方式）
```

Key: 动态申请和释放内存空间

C —— malloc、free 函数

L.data = (ElemType *) malloc(sizeof(ElemType) * InitSize);

C++ —— new、delete 关键字

malloc 函数返回一个指针，需要强制转型为你定义的数据元素类型指针

malloc 函数的参数，指明要分配多大的连续内存空间

内存



王道考研/CSKAOYAN.COM

8

malloc、free
函数的头文件

```
#include <stdlib.h>

#define InitSize 10 //默认的最大长度
typedef struct{
    int *data; //指示动态分配数组的指针
    int MaxSize; //顺序表的最大容量
    int length; //顺序表的当前长度
}SeqList;

int main() {
    SeqList L; //声明一个顺序表
    InitList(L); //初始化顺序表
    //...往顺序表中随便插入几个元素...
    IncreaseSize(L, 5);
    return 0;
}
```

注：realloc 函数也可实现，但建议初学者使用 malloc 和 free 更能理解背后过程

顺序表的实现——动态分配

```
void InitList(SeqList &L){
    //用 malloc 函数申请一片连续的存储空间
    L.data=(int *)malloc(InitSize*sizeof(int));
    L.length=0;
    L.MaxSize=InitSize;
}

//增加动态数组的长度
void IncreaseSize(SeqList &L, int len){
    int *p=L.data;
    L.data=(int *)malloc((L.MaxSize+len)*sizeof(int));
    for(int i=0; i<L.length; i++){
        L.data[i]=p[i];
    }
    L.MaxSize=L.MaxSize+len;
    free(p);
}
```

时间开销大

内存

王道考研/CSKAOYAN.COM

9

顺序表的实现

顺序表的特点：

- ①**随机访问**，即可以在 $O(1)$ 时间内找到第 i 个元素。
- ②存储密度高，每个节点只存储数据元素
- ③拓展容量不方便（即便采用动态分配的方式实现，拓展长度的时间复杂度也比较高）
- ④插入、删除操作不方便，需要移动大量元素

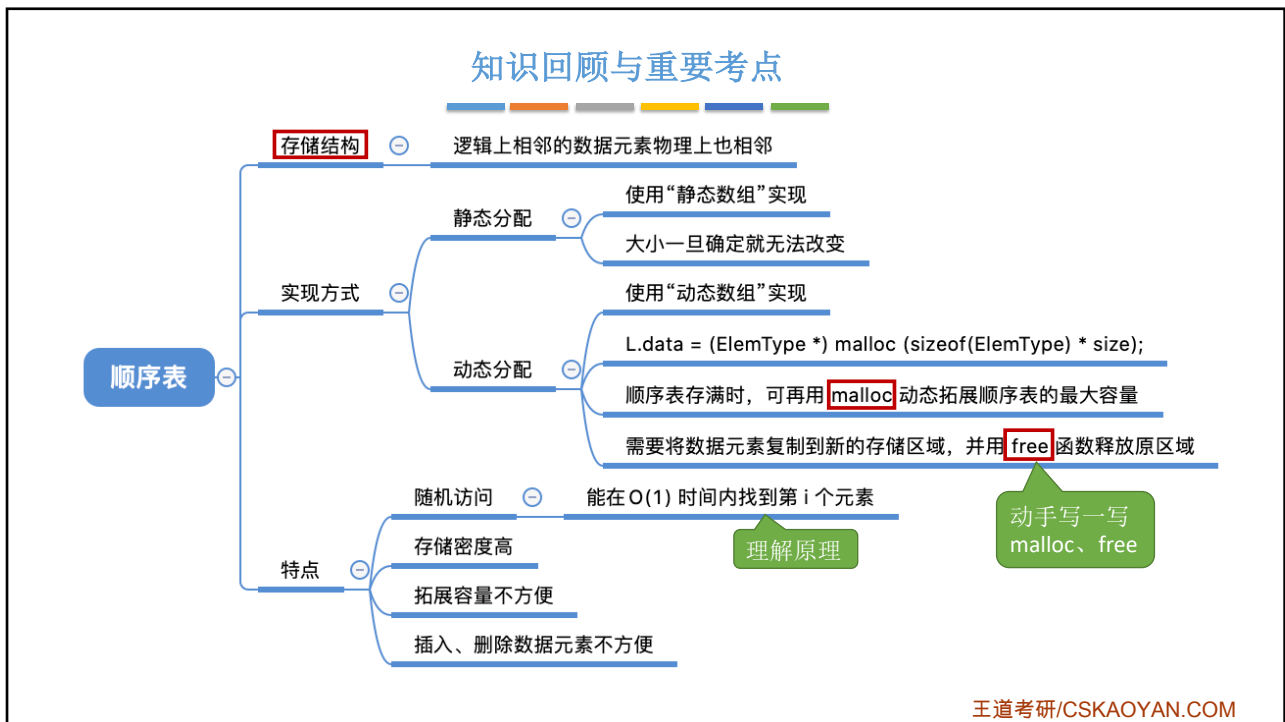
代码实现：data[i-1];
静态分配、动态分配都一样

王道考研/CSKAOYAN.COM

10

王道考研,cskaoyan.com

5



本节内容

顺序表
插入
删除

王道考研/CSKAOYAN.COM

1

知识总览

顺序表的插入和删除

插入

删除

代码实现

时间复杂度分析

代码实现

时间复杂度分析

王道考研/CSKAOYAN.COM

2

用存储位置的相邻来体现数据元素之间的逻辑关系

顺序表的基本操作——插入

位序

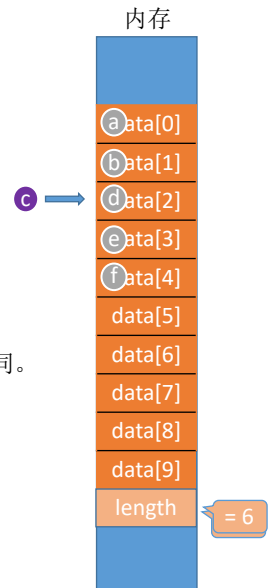
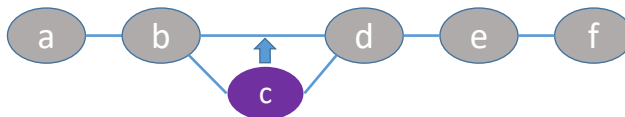
ListInsert(&L,i,e): 插入操作。在表L中的第i个位置上插入指定元素e。

```
#define MaxSize 10 //定义最大长度
typedef struct{
    ElemType data[MaxSize]; //用静态的“数组”存放数据元素
    int length; //顺序表的当前长度
}SqList; //顺序表的类型定义
```

用静态分配方式实现的顺序表

注：本节代码建立在顺序表的“静态分配”实现方式之上，“动态分配”也雷同。

逻辑结构:



王道考研/CSKAOYAN.COM

3

顺序表的基本操作——插入

```
#define MaxSize 10 //定义最大长度
typedef struct{
    int data[MaxSize]; //用静态的“数组”存放数据元素
    int length; //顺序表的当前长度
}SqList; //顺序表的类型定义
```

```
void ListInsert(SqList &L, int i, int e){
    for(int j=L.length; j>=i; j--) //将第i个元素及之后的元素后移
        L.data[j]=L.data[j-1];
    L.data[i-1]=e; //在位置i处放入e
    L.length++; //长度加1
}
```

基本操作：在L的位序 i 处插入元素e

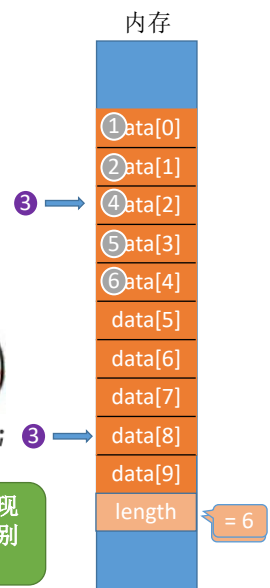
注意位序、数组下标的关系，并从后面的元素依次移动

ListInsert(L, 9, 3);

```
int main() {
    SqList L; //声明一个顺序表
    InitList(L); //初始化顺序表
    //...此处省略一些代码，插入几个元素
    ListInsert(L, 3, 3);
    return 0;
}
```

我可能就是天才吧

基操——让自己实现的数据结构可以让别人很方便地使用



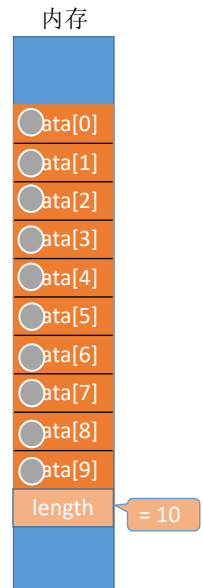
王道考研/CSKAOYAN.COM

4

顺序表的基本操作——插入

```
#define MaxSize 10    //定义最大长度
typedef struct{
    int data[MaxSize]; //用静态的“数组”存放数据元素
    int length;         //顺序表的当前长度
}SqList;               //顺序表的类型定义

bool ListInsert(SqList &L, int i, int e){
    if(i < 1 || i > L.length+1) //判断i的范围是否有效
        return false;
    if(L.length >= MaxSize)     //当前存储空间已满，不能插入
        return false;
    for(int j=L.length; j>=i; j--) //将第i个元素及之后的元素后移
        L.data[j]=L.data[j-1];
    L.data[i-1]=e;               //在位置i处放入e
    L.length++;                 //长度加1
    return true;
}
```



好的算法，应该具有“健壮性”。
能处理异常情况，并给使用者反馈

王道考研/CSKAOYAN.COM

5

插入操作的时间复杂度

```
bool ListInsert(SqList &L, int i, int e){
    if(i < 1 || i > L.length+1) //判断i的范围是否有效
        return false;
    if(L.length >= MaxSize)     //当前存储空间已满，不能插入
        return false;
    for(int j=L.length; j>=i; j--) //将第i个元素及之后的元素后移
        L.data[j]=L.data[j-1];
    L.data[i-1]=e;               //在位置i处放入e
    L.length++;                 //长度加1
    return true;
}
```

关注最深层循环语句的执行
次数与问题规模 n 的关系

问题规模 n = L.length (表长)

最好情况：新元素插入到表尾，不需要移动元素
i = n+1，循环0次；最好时间复杂度 = O(1)
最坏情况：新元素插入到表头，需要将原有的 n 个元素全都向后移动
i = 1，循环 n 次；最坏时间复杂度 = O(n);
平均情况：假设新元素插入到任何一个位置的概率相同，即 i = 1, 2, 3, ..., length+1 的概率都是 $p = \frac{1}{n+1}$
i = 1，循环 n 次；i = 2 时，循环 n-1 次；i = 3，循环 n-2 次 i = n+1 时，循环 0 次
平均循环次数 = $np + (n-1)p + (n-2)p + \dots + 1 \cdot p = \frac{n(n+1)}{2} \cdot \frac{1}{n+1} = \frac{n}{2}$ → 平均时间复杂度 = O(n)

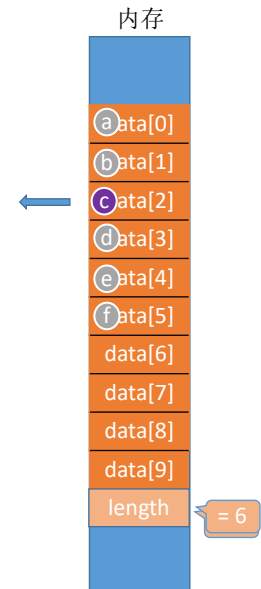
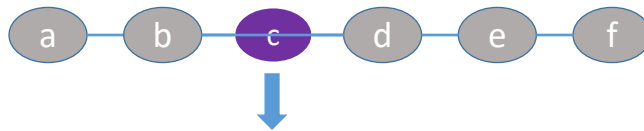
王道考研/CSKAOYAN.COM

6

顺序表的基本操作——删除

ListDelete(&L,i,&e): 删除操作。删除表L中第i个位置的元素，并用e返回删除元素的值。

逻辑结构:



王道考研/CSKAOYAN.COM

7

顺序表的基本操作——删除

```
bool ListDelete(SqList &L, int i, int &e){
    if(i<1||i>L.length) //判断i的范围是否有效
        return false;
    e=L.data[i-1]; //将被删除的元素赋值给e
    for(int j=i;j<L.length;j++) //将第i个位置后的元素前移
        L.data[j-1]=L.data[j];
    L.length--; //线性表长度减1
    return true;
}

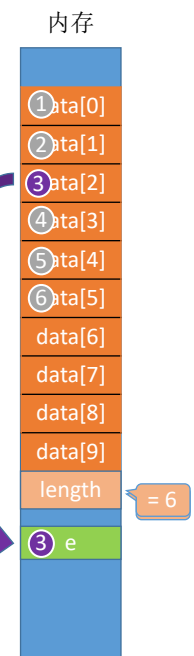
int main() {
    SqList L; //声明一个顺序表
    InitList(L); //初始化顺序表
    //...此处省略一些代码，插入几个元素
    int e = -1; //用变量e把删除的元素“带回来”
    if (ListDelete(L, 3, e))
        printf("已删除第3个元素,删除元素值为%d\n", e);
    else
        printf("位序i不合法,删除失败\n");
    return 0;
}
```

注意位序、数组下标的关系，并从前面的元素依次移动

如果参数没有加引用符号，会怎样？



复制



已删除第3个元素,删除元素值为3
Program ended with exit code: 0

王道考研/CSKAOYAN.COM

8

删除操作的时间复杂度

```
bool ListDelete(SqList &L, int i, int &e){
    if(i<1||i>L.length) //判断i的范围是否有效
        return false;
    e=L.data[i-1]; //将被删除的元素赋值给e
    for(int j=i;j<L.length;j++) //将第i个位置后的元素前移
        L.data[j-1]=L.data[j];
    L.length--; //
    return true;
}
```

关注最深层循环语句的执行次数与问题规模 n 的关系

问题规模 $n = L.length$ (表长)

最好情况：删除表尾元素，不需要移动其他元素

$i = n$ ，循环 0 次；最好时间复杂度 = $O(1)$

最坏情况：删除表头元素，需要将后续的 $n-1$ 个元素全都向前移动

$i = 1$ ，循环 $n-1$ 次；最坏时间复杂度 = $O(n)$ ；

平均情况：假设删除任何一个元素的概率相同，即 $i = 1, 2, 3, \dots, length$ 的概率都是 $p = \frac{1}{n}$

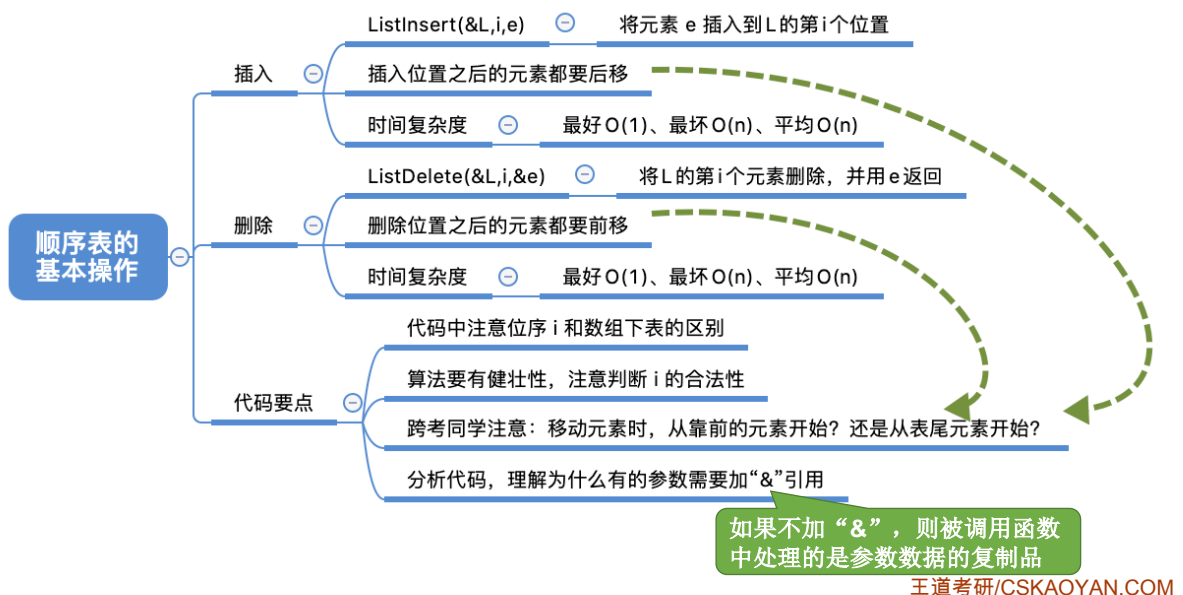
$i = 1$ ，循环 $n-1$ 次； $i = 2$ 时，循环 $n-2$ 次； $i = 3$ ，循环 $n-3$ 次 $i = n$ 时，循环 0 次

平均循环次数 = $(n-1)p + (n-2)p + \dots + 1 \cdot p = \frac{n(n-1)}{2} \cdot \frac{1}{n} = \frac{n-1}{2}$ → 平均时间复杂度 = $O(n)$

王道考研/CSKAOYAN.COM

9

知识回顾与重要考点



10

本节内容

顺序表
查找

王道考研/CSKAOYAN.COM

1

知识总览

顺序表的基本操作

按位查找

按值查找

代码实现

时间复杂度分析

代码实现

时间复杂度分析

王道考研/CSKAOYAN.COM

2

顺序表的按位查找

GetElem(L,i): 按位查找操作。获取表L中第i个位置的元素的值。

```
#define MaxSize 10           //定义最大长度
typedef struct{
    ElemType data[MaxSize];  //用静态的“数组”存放数据元素
    int length;              //顺序表的当前长度
}SqList;                     //顺序表的类型定义（静态分配方式）

ElemType GetElem(SqList L, int i){
    return L.data[i-1];
}
```

静态分配

王道考研/CSKAOYAN.COM

3

顺序表的按位查找

GetElem(L,i): 按位查找操作。获取表L中第i个位置的元素的值。

```
#define InitSize 10          //顺序表的初始长度
typedef struct{
    ElemType *data;           //指示动态分配数组的指针
    int MaxSize;              //顺序表的最大容量
    int length;               //顺序表的当前长度
} SeqList;                   //顺序表的类型定义（动态分配方式）

ElemType GetElem(SeqList L, int i){
    return L.data[i-1];
}
```

动态分配

和访问普通数组的方法一样

ElemType *data 如果一个 ElemType 占 6B，即 sizeof(ElemType)==6
指针 data 指向的地址为 2000



王道考研/CSKAOYAN.COM

4

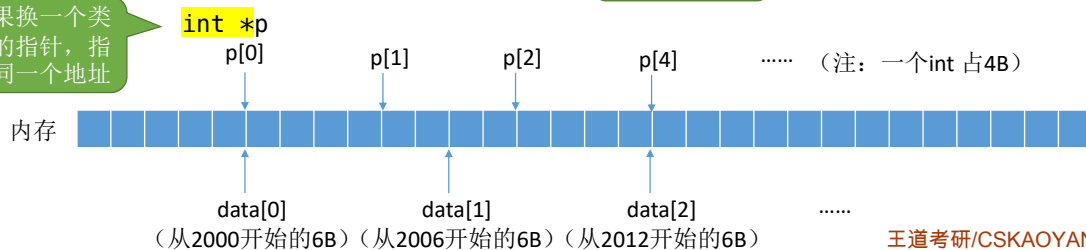
顺序表的按位查找

GetElem(L,i): 按位查找操作。获取表L中第i个位置的元素的值。

```
#define InitSize 10          //顺序表的初始长度
typedef struct{
    ElemType *data;          //指示动态分配数组的指针 动态分配
    int MaxSize;             //顺序表的最大容量
    int length;              //顺序表的当前长度
} SeqList;                  //顺序表的类型定义（动态分配方式）
```

```
ElemType GetElem(SeqList L, int i){
    return L.data[i-1];      //和访问普通数组的方法一样
```

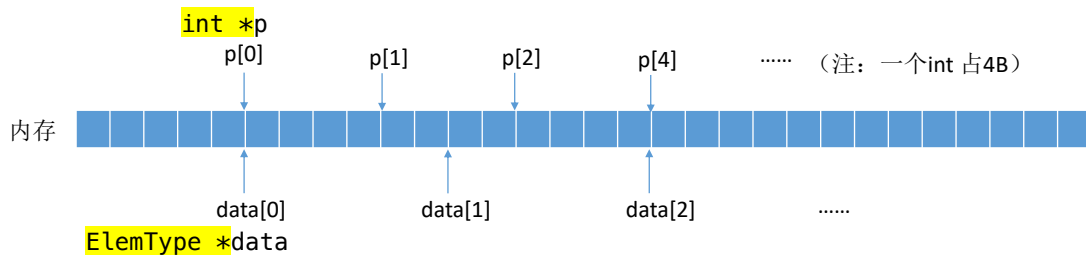
如果换一个类型的指针，指向同一个地址



王道考研/CSKAOYAN.COM

5

顺序表的按位查找



```
#define InitSize 10 //默认的最大长度
typedef struct{
    int *data;        //指示动态分配数组的指针
    int MaxSize;      //顺序表的最大容量
    int length;       //顺序表的当前长度
} SeqList;
```

```
void InitList(SeqList &L){
    //用 malloc 函数申请一片连续的存储空间
    L.data=(int *)malloc(InitSize*sizeof(int));
    L.length=0;
    L.MaxSize=InitSize;
}
```

再次理解，为何malloc函数返回的存储空间起始地址要转换为与数据元素的数据类型相对应的指针

王道考研/CSKAOYAN.COM

6

按位查找的时间复杂度



```
ElemType GetElem(SeqList L, int i){
    return L.data[i-1];
}
```

时间复杂度: $O(1)$

由于顺序表的各个数据元素在内存中连续存放，因此可以根据起始地址和数据元素大小立即找到第 i 个元素——“随机存取”特性

王道考研/CSKAOYAN.COM

7

顺序表的按值查找

LocateElem(L,e): 按值查找操作。在表L中查找具有给定关键字值的元素。

```
#define InitSize 10          //顺序表的初始长度
typedef struct{
    ElemType *data;          //指示动态分配数组的指针
    int MaxSize;              //顺序表的最大容量
    int length;               //顺序表的当前长度
} SeqList;                   //顺序表的类型定义（动态分配方式）

//在顺序表L中查找第一个元素值等于e的元素，并返回其位序
int LocateElem(SeqList L, ElemType e){
    for(int i=0; i<L.length; i++){
        if(L.data[i]==e)
            return i+1;      //数组下标为i的元素值等于e，返回其位序i+1
    }
    return 0;                 //退出循环，说明查找失败
}
```

王道考研/CSKAOYAN.COM

8

顺序表的按值查找

LocateElem(L,e): 按值查找操作。在表L中查找具有给定关键字值的元素。

```
typedef struct{
    int *data;           //指示动态分配数组的指针
    int MaxSize;         //顺序表的最大容量
    int length;          //顺序表的当前长度
}SeqList;
```

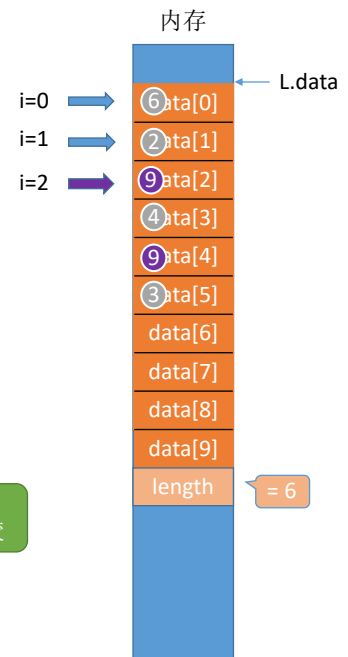
//在顺序表L中查找第一个元素值等于e的元素，并返回其位序

```
int LocateElem(SeqList L,int e){
    for(int i=0;i<L.length;i++){
        if(L.data[i]==e)
            return i+1;
    }
    return 0;
}
```

调用: LocateElem(L, 9);

基本数据类型: int、char、double、float 等可以直接用运算符“==”比较

结构类型的数据元素也这样吗?



王道考研/CSKAOYAN.COM

9

结构类型的比较

```
typedef struct {
    int num;
    int people;
} Customer;
```

```
void test () {
    Customer a;
    a.num = 1;
    a.people = 1;
    Customer b;
    b.num = 1;
    b.people = 1;
    if (a == b) {
        printf("相等");
    }else {
        printf("不相等");
    }
}
```



不能

```
bool isCustomerEqual (Customer a, Customer b){
    if (a.num == b.num && a.people == b.people)
        return true;
    else
        return false;
}
```

更好的办法: 定义一个函数, 童叟无欺, 用过都说好

注意: C语言中, 结构体的比较不能直接用“==”

需要依次对比各个分量来判断两个结构体是否相等

```
if (a.num == b.num && a.people == b.people) {
    printf("相等");
}else {
    printf("不相等");
}
```

王道考研/CSKAOYAN.COM

10

顺序表的按值查找

LocateElem(L,e): **按值查找**操作。在表L中查找具有给定关键字值的元素。

```
//在顺序表L中查找第一个元素值等于e的元素，并返回其位序
int LocateElem(SeqList L,ElemType e){
    for(int i=0;i<L.length;i++)
        if(L.data[i]==e)
            return i+1;    //数组下标为i的元素值等于e，返回其位序i+1
    return 0;              //退出循环，说明查找失败
}
```

Tips:

《数据结构》考研初试中，手写代码可以直接用“==”，无论 ElemType 是基本数据类型还是结构类型

手写代码主要考察学生是否能理解算法思想，不会严格要求代码完全可运行

有的学校考《C语言程序设计》，那么...也许就要语法严格一些

王道考研/CSKAOYAN.COM

11

按值查找的时间复杂度

```
//在顺序表L中查找第一个元素值等于e的元素，并返回其位序
int LocateElem(SeqList L,ElemType e){
    for(int i=0;i<L.length;i++)
        if(L.data[i]==e)
            return i+1;
    return 0;
}
```

关注最深层循环语句的执行次数与问题规模 n 的关系
返回其位序 i+1
问题规模 n = L.length (表长)

最好情况：目标元素在表头
循环1次；最好时间复杂度 = $O(1)$

最坏情况：目标元素在表尾
循环 n 次；最坏时间复杂度 = $O(n)$;

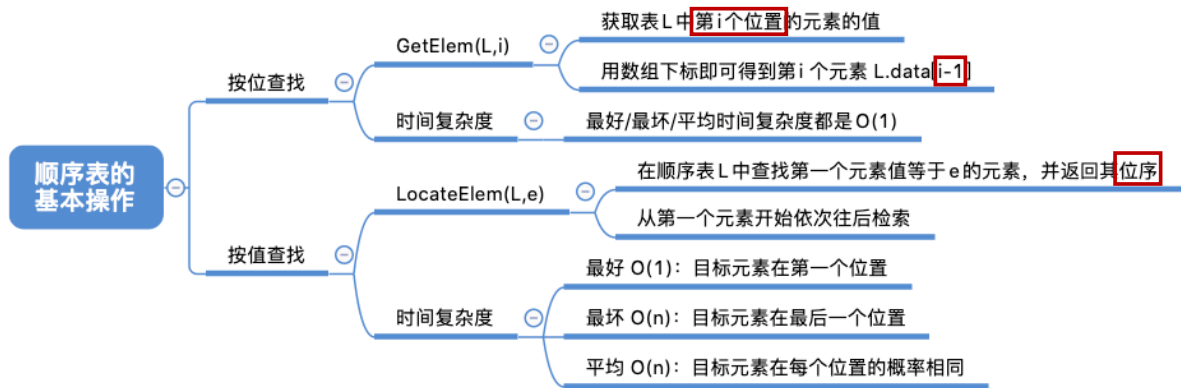
平均情况：假设目标元素出现在任何一个位置的概率相同，都是 $\frac{1}{n}$
目标元素在第1位，循环1次；在第2位，循环2次；.....；在第 n 位，循环 n 次

$$\text{平均循环次数} = 1 \cdot \frac{1}{n} + 2 \cdot \frac{1}{n} + 3 \cdot \frac{1}{n} + \dots + n \cdot \frac{1}{n} = \frac{n(n+1)}{2} \cdot \frac{1}{n} = \frac{n+1}{2} \Rightarrow \text{平均时间复杂度} = O(n)$$

王道考研/CSKAOYAN.COM

12

知识回顾与重要考点



王道考研/CSKAOYAN.COM

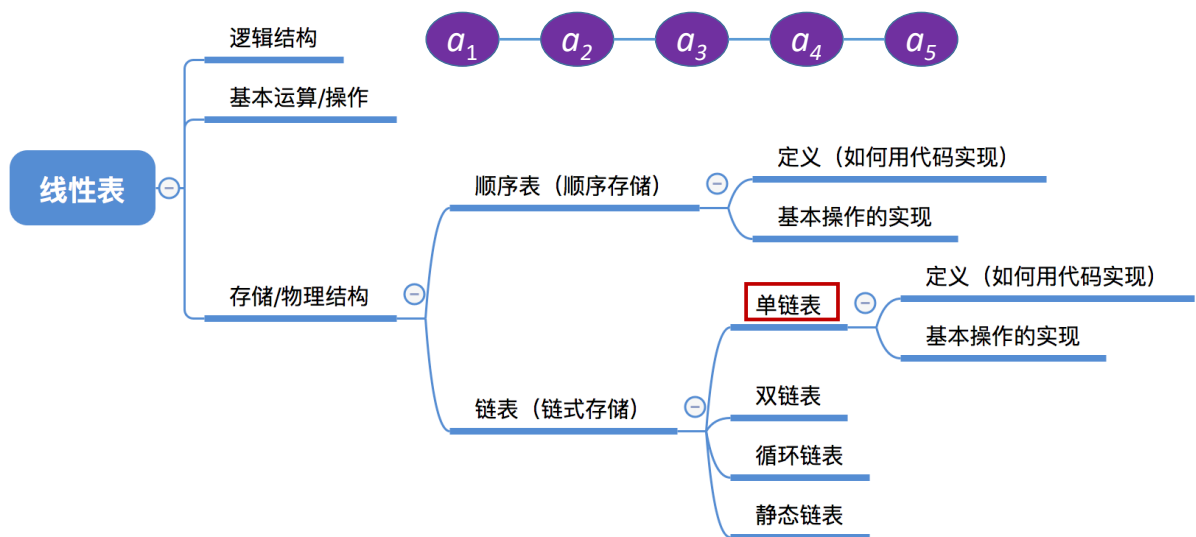
本节内容

单链表 定义

王道考研/CSKAOYAN.COM

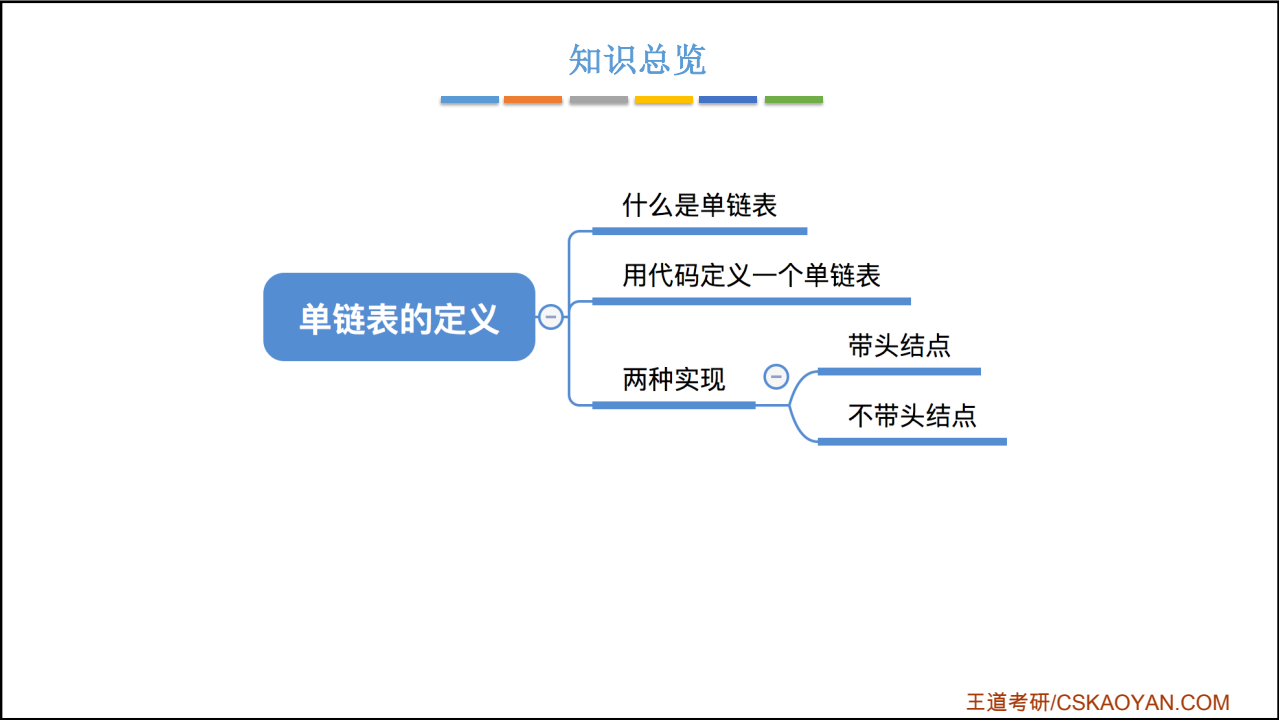
1

知识总览

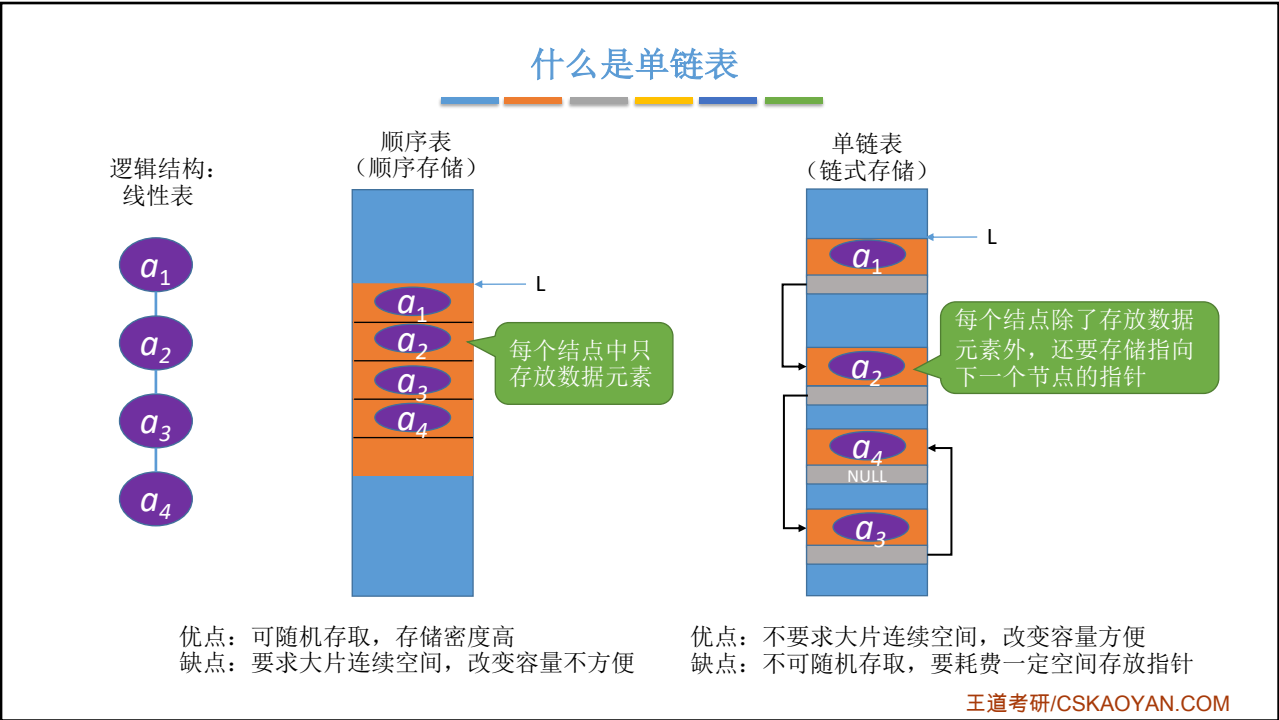


王道考研/CSKAOYAN.COM

2

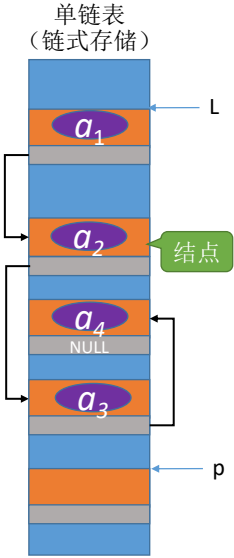


3



4

单链表
(链式存储)



用代码定义一个单链表

结点

```
struct LNode{  
    ElemType data;  
    struct LNode *next;  
};
```

数据域

指针域

//定义单链表结点类型
//每个节点存放一个数据元素
//指针指向下一个节点

```
struct LNode * p = (struct LNode *) malloc(sizeof(struct LNode));
```

增加一个新的结点：在内存中申请一个结点所需空间，并用指针 p 指向这个结点

事情并不简单

typedef 关键字 —— 数据类型重命名

```
typedef <数据类型> <别名>  
typedef int zhengshu;  
typedef int *zhengshuzhizhen;  
int x = 1;  
int *p;
```

巴啦啦能量

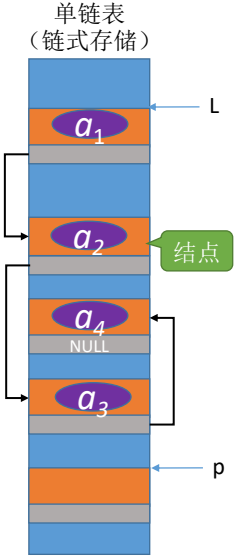
等价

```
zhengshu x = 1;  
zhengshuzhizhen p;
```

王道考研/CSKAOYAN.COM

5

单链表
(链式存储)



用代码定义一个单链表

结点

```
struct LNode{  
    ElemType data;  
    struct LNode *next;  
};
```

数据域

指针域

//定义单链表结点类型
//每个节点存放一个数据元素
//指针指向下一个节点

```
struct LNode * p = (struct LNode *) malloc(sizeof(struct LNode));
```

增加一个新的结点：在内存中申请一个结点所需空间，并用指针 p 指向这个结点

事情并不简单

typedef 关键字 —— 数据类型重命名

```
typedef <数据类型> <别名>  
typedef struct LNode LNode;  
LNode * p = (LNode *) malloc(sizeof(LNode));
```

巴啦啦能量

原来如此，简单!

王道考研/CSKAOYAN.COM

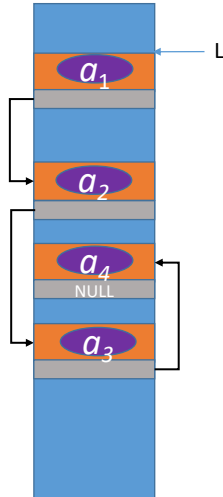
6

王道考研/CSKAOYAN.COM

3

用代码定义一个单链表

单链表
(链式存储)



```
typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;
```

//定义单链表结点类型
//每个节点存放一个数据元素
//指针指向下一个节点

```
struct LNode{
    ElemType data;
    struct LNode *next;
};
```

//定义单链表结点类型
//每个节点存放一个数据元素
//指针指向下一个节点

```
typedef struct LNode LNode;
typedef struct LNode *LinkList;
```

要表示一个单链表时, 只需声明一个头指针 L , 指向单链表的第一个结
 $LNode * L;$ //声明一个指向单链表第一个结点的指针

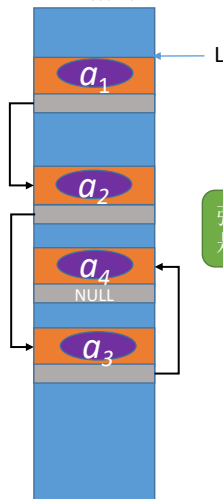
或: $LinkList L;$ //声明一个指向单链表第一个结点的指针 代码可读性更强

王道考研/CSKAOYAN.COM

7

用代码定义一个单链表

单链表
(链式存储)



```
typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;
```

//定义单链表结点类型
//每个节点存放一个数据元素
//指针指向下一个节点

```
LNode * GetElem(LinkList L, int i){
    int j=1;
    LNode *p=L->next;
    if(i==0)
        return L;
    if(i<1)
        return NULL;
    while(p!=NULL && j<i){
        p=p->next;
        j++;
    }
    return p;
}
```

强调返回的是一个结点
强调这是一个单链表

强调这是一个单链表
强调这是一个结点
——使用 LinkList
——使用 LNode *

王道考研/CSKAOYAN.COM

8

用代码定义一个单链表

头插法建立单链表的算法如下：

```
LinkedList List_HeadInsert(LinkedList &L) { // 逆向建立单链表
    LNode *s; int x;
    L = (LinkedList)malloc(sizeof(LNode)); // 创建头结点
    L->next = NULL; // 初始为空链表
    scanf("%d", &x); // 输入结点的值
    while(x != 9999) { // 输入 9999 表示结束
        s = (LNode*)malloc(sizeof(LNode)); // 创建新结点
        s->data = x;
        s->next = L->next;
        L->next = s; // 将新结点插入表中，L 为头指针
        scanf("%d", &x);
    }
    return L;
}
```

强调这是一个单链表
强调这是一个结点

——使用 LinkedList
——使用 LNode *

王道考研/CSKAOYAN.COM

9

不带头结点的单链表

```
typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkedList;
```

// 定义单链表结点类型
// 每个结点存放一个数据元素
// 指针指向下一个节点

// 初始化一个空的单链表

```
bool InitList(LinkedList &L) {
```

→ L = NULL; // 空表，暂时还没有任何结点

return true;

}

```
void test(){
```

→ LinkedList L; // 声明一个指向单链表的指针

// 初始化一个空表

→ InitList(L);

//后续代码.....

}

注意，此处
并没有创建
一个结点

防止脏数据

// 判断单链表是否为空

```
bool Empty(LinkedList L) {
```

if (L == NULL)

return true;

else

return false;

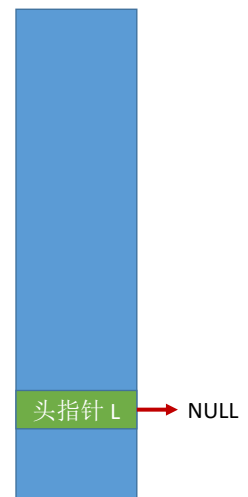
}

或：bool Empty(LinkedList L) {

return (L == NULL);

}

内存



王道考研/CSKAOYAN.COM

10

带头结点的单链表

```

typedef struct LNode{           //定义单链表结点类型
    ElemType data;             //每个节点存放一个数据元素
    struct LNode *next;        //指针指向下一个节点
}LNode, *LinkList;

//初始化一个单链表（带头结点）
bool InitList(LinkList &L) {
    L = (LNode *) malloc(sizeof(LNode)); //分配一个头结点
    if (L==NULL)                          //内存不足，分配失败
        return false;
    L->next = NULL;                        //头结点之后暂时还没有节点
    return true;
}

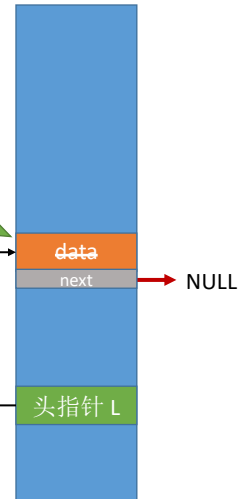
//判断单链表是否为空（带头结点）
bool Empty(LinkList L) {
    if (L->next == NULL)
        return true;
    else
        return false;
}

void test(){
    LinkList L; //声明一个指向单链表的指针
    //初始化一个空表
    InitList(L);
    //.....后续代码.....
}

```

头结点不
存储数据

内存

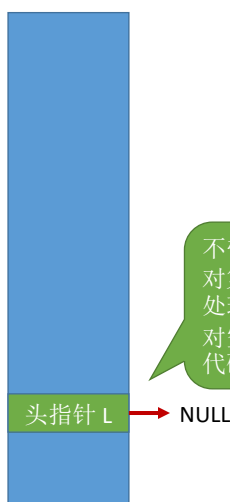


王道考研/CSKAOYAN.COM

11

不带头结点 v.s. 带头结点

不带头

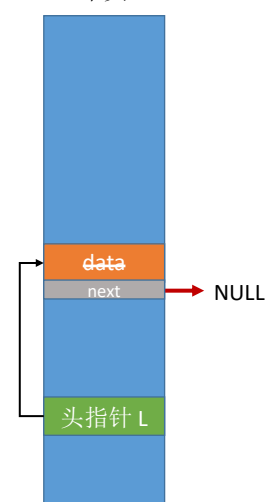


带头结点，写代码更
方便，用过都说好



不带头结点，写代码更麻烦
对第一个数据结点和后续数据结点的
处理需要用不同的代码逻辑
对空表和非空表的处理需要用不同的
代码逻辑

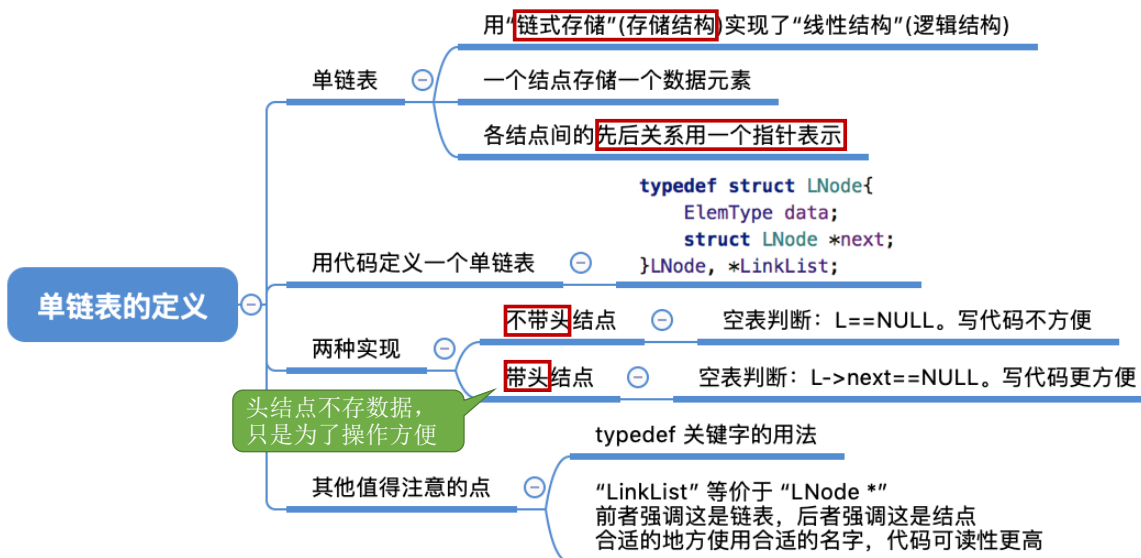
带头



王道考研/CSKAOYAN.COM

12

知识回顾与重要考点



王道考研/CSKAOYAN.COM

本节内容

单链表
插入和删除

王道考研/CSKAOYAN.COM

1

知识总览

单链表的插入删除

插入

按位序插入

指定结点的后插操作

指定结点的前插操作

删除

按位序删除

指定结点的删除

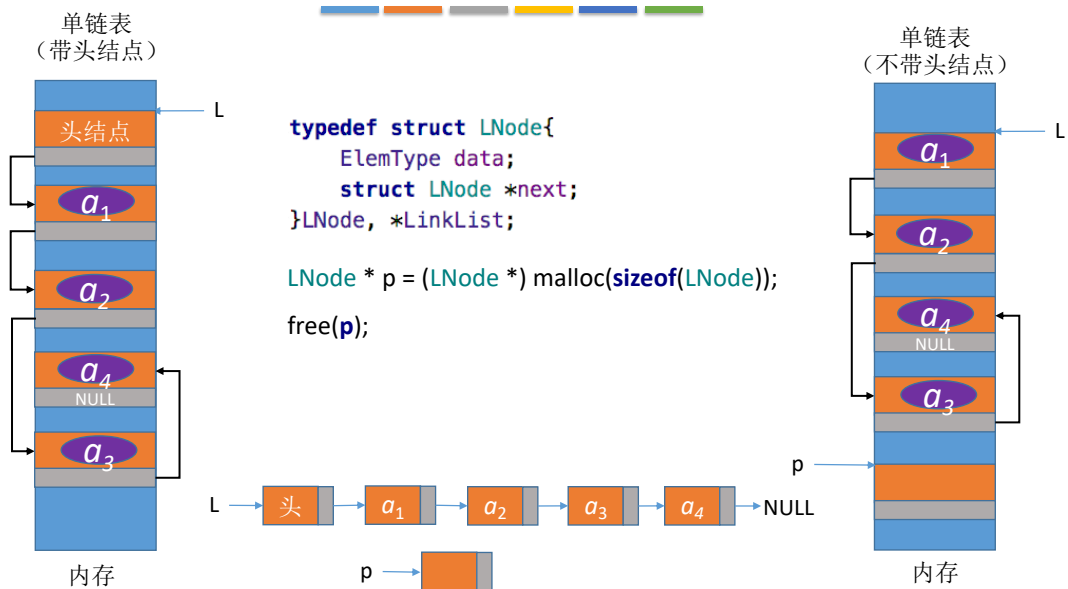
带头结点

不带头结点

王道考研/CSKAOYAN.COM

2

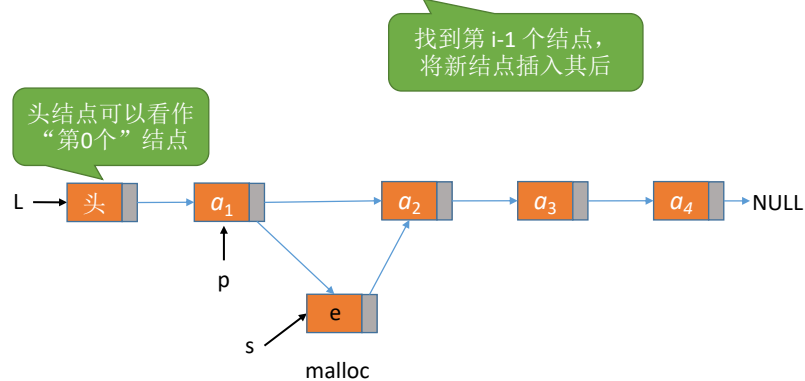
关于简化图示的说明



3

按位序插入 (带头结点)

ListInsert(&L,i,e): 插入操作。在表L中的第i个位置上插入指定元素e。



4

按位序插入（带头结点）

```

//在第 i 个位置插入元素 e (带头结点)
bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
    LNode *p; //指针p指向当前扫描到的结点
    int j=0; //当前p指向的是第几个结点
    p = L; //L指向头结点, 头结点是第0个结点 (不存数据)
    while (p!=NULL && j<i-1) { //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    if(p==NULL) //i值不合法
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = e;
    s->next=p->next;
    p->next=s; //将结点s连到p之后
    return true; //插入成功
}

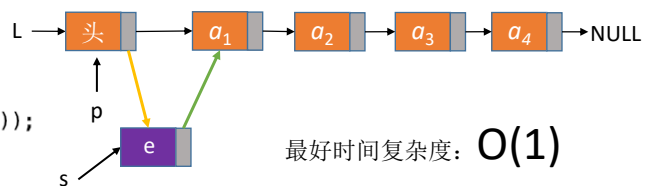
```

```

typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;

```

分析:
①如果 $i = 1$ (插在表头)



最好时间复杂度: $O(1)$

注意: 绿绿和黄黄顺序不能颠倒鸭!

王道考研/CSKAOYAN.COM

5

按位序插入（带头结点）

```

//在第 i 个位置插入元素 e (带头结点)
bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
    LNode *p; //指针p指向当前扫描到的结点
    int j=0; //当前p指向的是第几个结点
    p = L; //L指向头结点, 头结点是第0个结点 (不存数据)
    while (p!=NULL && j<i-1) { //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    if(p==NULL) //i值不合法
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = e;
    s->next=p->next;
    p->next=s; //将结点s连到p之后
    return true; //插入成功
}

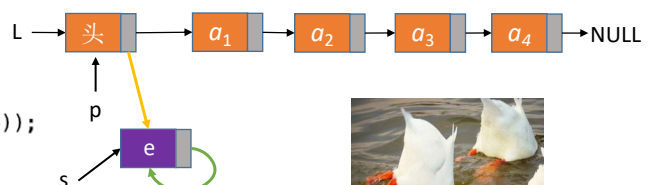
```

```

typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;

```

分析:
①如果 $i = 1$ (插在表头)



注意: 绿绿和黄黄顺序不能颠倒鸭!

王道考研/CSKAOYAN.COM

6

按位序插入（带头结点）

```

//在第 i 个位置插入元素 e (带头结点)
bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
    LNode *p;    //指针p指向当前扫描到的结点
    int j=0;      //当前p指向的是第几个结点
    p = L;       //L指向头结点，头结点是第0个结点（不存数据）
    while (p!=NULL && j<i-1) { //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    if(p==NULL)    //i值不合法
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = e;
    s->next=p->next;
    p->next=s;      //将结点s连到p之后
    return true;    //插入成功
}

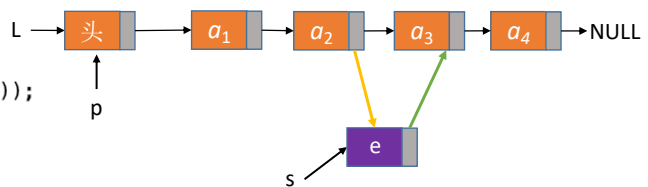
```

```

typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;

```

分析：
②如果 $i=3$ （插在表中）



王道考研/CSKAOYAN.COM

7

按位序插入（带头结点）

```

//在第 i 个位置插入元素 e (带头结点)
bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
    LNode *p;    //指针p指向当前扫描到的结点
    int j=0;      //当前p指向的是第几个结点
    p = L;       //L指向头结点，头结点是第0个结点（不存数据）
    while (p!=NULL && j<i-1) { //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    if(p==NULL)    //i值不合法
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = e;
    s->next=p->next;
    p->next=s;      //将结点s连到p之后
    return true;    //插入成功
}

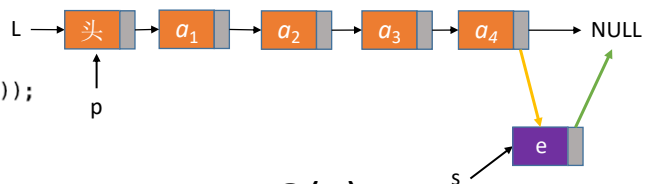
```

```

typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;

```

分析：
③如果 $i=5$ （插在表尾）



最坏时间复杂度: $O(n)$

王道考研/CSKAOYAN.COM

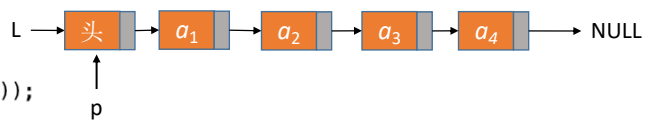
8

按位序插入（带头结点）

```
//在第 i 个位置插入元素 e (带头结点)
bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
    LNode *p; //指针p指向当前扫描到的结点
    int j=0; //当前p指向的是第几个结点
    p = L; //L指向头结点，头结点是第0个结点（不存数据）
    while (p!=NULL && j<i-1) { //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    if(p==NULL) //i值不合法
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = e;
    s->next=p->next;
    p->next=s; //将结点s连到p之后
    return true; //插入成功
}
```

```
typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;
```

分析：
④如果 $i = 6$ ($i > \text{Lengh}$)



王道考研/CSKAOYAN.COM

9

按位序插入（带头结点）

```
//在第 i 个位置插入元素 e (带头结点)
bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
    LNode *p; //指针p指向当前扫描到的结点
    int j=0; //当前p指向的是第几个结点
    p = L; //L指向头结点，头结点是第0个结点（不存数据）
    while (p!=NULL && j<i-1) { //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    if(p==NULL) //i值不合法
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = e;
    s->next=p->next;
    p->next=s; //将结点s连到p之后
    return true; //插入成功
}
```

```
typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;
```

平均时间复杂度: $O(n)$

原来如此，简单！

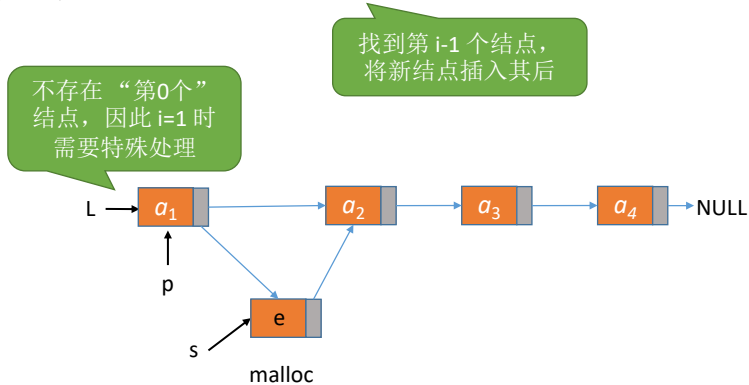


王道考研/CSKAOYAN.COM

10

按位序插入（不带头结点）

ListInsert(&L,i,e): 插入操作。在表L中的第*i*个位置上插入指定元素e。



王道考研/CSKAOYAN.COM

11

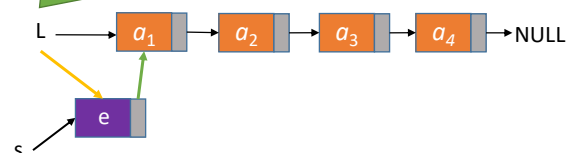
按位序插入（不带头结点）

```
bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
    if(i==1){ //插入第1个结点的操作与其他结点操作不同
        LNode *s = (LNode *)malloc(sizeof(LNode));
        s->data = e;
        s->next=L;
        L=s; //头指针指向新结点
        return true;
    }
    LNode *p; //指针p指向当前扫描到的结点
    int j=1; //当前p指向的是第几个结点
    p = L; //p指向第1个结点（注意：不是头结点）
    while (p!=NULL && j<i-1) { //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    if(p==NULL) //i值不合法
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = e;
    s->next=p->next;
    p->next=s;
    return true; //插入成功
}
```

```
typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;
```

分析：
①如果 $i = 1$ （插在表头）

如果不带头结点，则插入、删除第1个元素时，需要更改头指针L



王道考研/CSKAOYAN.COM

12

按位序插入（不带头结点）

```

bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
    if(i==1){ //插入第1个结点的操作与其他结点操作不同
        LNode *s = (LNode *)malloc(sizeof(LNode));
        s->data = e;
        s->next=L;
        L=s; //头指针指向新结点
        return true;
    }
    LNode *p; //指针p指向当前扫描到的结点
    int j=1; //当前p指向的是第几个结点
    p = L; //p指向第1个结点（注意：不是头结点）
    while (p!=NULL && j<i-1) { //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    if(p==NULL) //i值不合法
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = e;
    s->next=p->next;
    p->next=s;
    return true; //插入成功
}

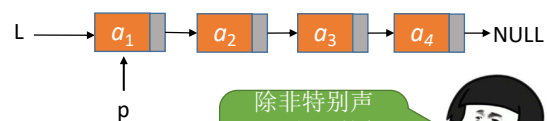
```

```

typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;

```

分析：
②如果 $i > 1...$



后续逻辑和带头结点的一样

除非特别声明，之后的代码默认带头结点



结论：不带头结点写代码更不方便，推荐用带头结点
注意：考试中带头、不带头都有可能考察，注意审题

王道考研/CSKAOYAN.COM

13

指定结点的后插操作

```

//后插操作：在p结点之后插入元素 e
bool InsertNextNode (LNode *p, ElemType e){
    if (p==NULL)
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    if (s==NULL) //内存分配失败
        return false;
    s->data = e; //用结点s保存数据元素e
    s->next=p->next;
    p->next=s; //将结点s连到p之后
    return true;
}

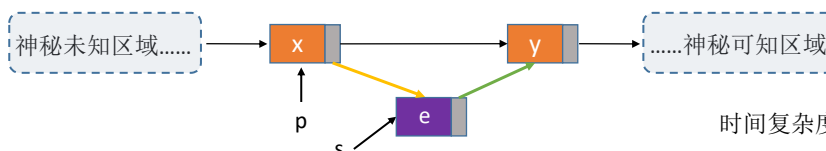
```

```

typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;

```

某些情况下有可能分配失败（如内存不足）



时间复杂度： $O(1)$

王道考研/CSKAOYAN.COM

14

指定结点的后插操作

```
//在第 i 个位置插入元素 e (带头结点)
bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
    LNode *p;    //指针p指向当前扫描到的结点
    int j=0;      //当前p指向的是第几个结点
    p = L;       //L指向头结点, 头结点是第0个结点 (不存数据)
    while (p!=NULL && j<i-1) { //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    if(p==NULL) //i值不合法
        return InsertNextNode(p, e);
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = e;
    s->next=p->next;
    p->next=s;    //将结点s连到p之后
    return true; //插入成功
}
```

```
typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;
```

```
//后插操作: 在p结点之后插入元素 e
bool InsertNextNode (LNode *p, ElemType e){
    if (p==NULL)
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    if (s==NULL) //内存分配失败
        return false;
    s->data = e;    //用结点s保存数据元素e
    s->next=p->next;
    p->next=s;      //将结点s连到p之后
    return true;
}
```

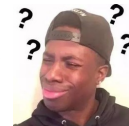
王道考研/CSKAOYAN.COM

15

指定结点的前插操作

```
//前插操作: 在p结点之前插入元素 e
bool InsertPriorNode (LNode *p, ElemType e)
```

如何找到 p 结点的前驱结点?

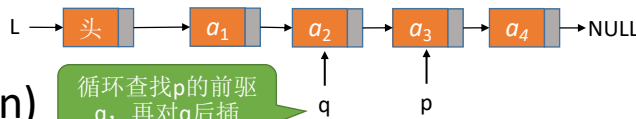


笑容被打上马赛克



```
//前插操作: 在p结点之前插入元素 e
bool InsertPriorNode (LinkList L, LNode *p, ElemType e)
```

传入头指针



高清无码
拒绝神秘

时间复杂度: $O(n)$

循环查找p的前驱q, 再对q后插

王道考研/CSKAOYAN.COM

16

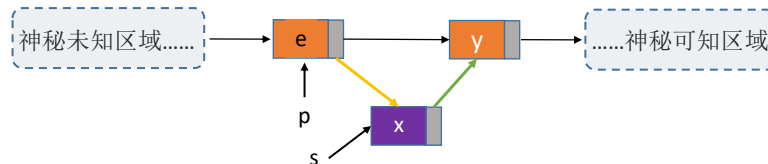
指定结点的前插操作

//前插操作：在p结点之前插入元素 e

```
bool InsertPriorNode (LNode *p, ElemType e){
    if (p==NULL)
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    if (s==NULL) //内存分配失败
        return false;
    s->next=p->next;
    p->next=s;
    s->data=p->data;
    p->data=e;
    return true;
}
```



时间复杂度: $O(1)$



王道考研/CSKAOYAN.COM

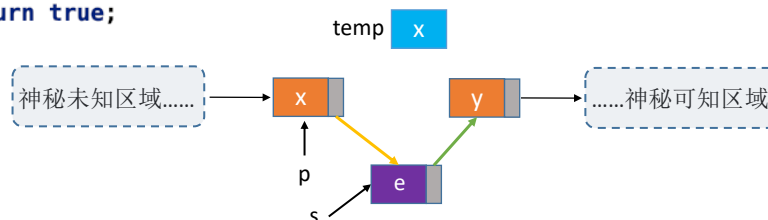
17

指定结点的前插操作

//前插操作：在p结点之前插入结点 s

```
bool InsertPriorNode (LNode *p, LNode *s){
    if (p==NULL || s==NULL)
        return false;
    s->next=p->next;
    p->next=s;
    ElemType temp=p->data; //交换数据域部分
    p->data=s->data;
    s->data=temp;
    return true;
}
```

王道书版本

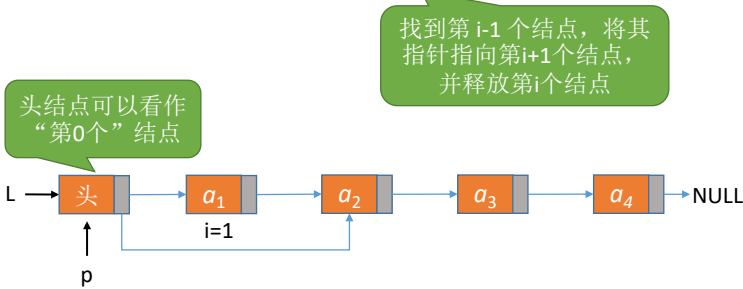


王道考研/CSKAOYAN.COM

18

按位序删除（带头结点）

ListDelete(&L,i,&e): 删除操作。删除表L中第i个位置的元素，并用e返回删除元素的值。



王道考研/CSKAOYAN.COM

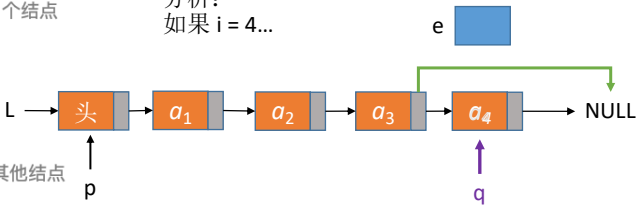
19

按位序删除（带头结点）

```
bool ListDelete(LinkList &L, int i, ElemType &e){
    if(i<1)
        return false;
    LNode *p; //指针p指向当前扫描到的结点
    int j=0; //当前p指向的是第几个结点
    p = L; //L指向头结点，头结点是第0个结点（不存数据）
    while (p!=NULL && j<i-1) { //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    if(p==NULL) //i值不合法
        return false;
    if(p->next == NULL) //第i-1个结点之后已无其他结点
        return false;
    LNode *q=p->next; //令q指向被删除结点
    e = q->data; //用e返回元素的值
    p->next=q->next; //将*q结点从链中“断开”
    free(q); //释放结点的存储空间
    return true; //删除成功
}
```

```
typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;
```

分析：
如果 i = 4...



最坏、平均时间复杂度: $O(n)$
最好时间复杂度: $O(1)$

如果不带头结点，删除第1个元素，是否需要特殊处理？

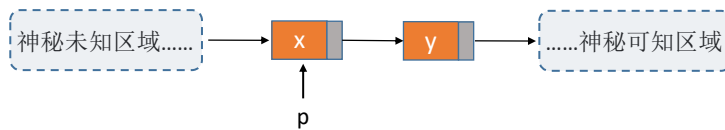


王道考研/CSKAOYAN.COM

20

指定结点的删除

//删除指定结点 p
bool DeleteNode (LNode *p)



我这暴脾气

删除结点 p ，需要修改其前驱结点的 next 指针

方法1：传入头指针，循环寻找 p 的前驱结点
 方法2：偷天换日（类似于结点前插的实现）

王道考研/CSKAOYAN.COM

21

指定结点的删除

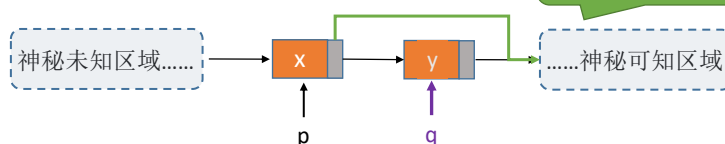
```

//删除指定结点 p
bool DeleteNode (LNode *p){
    if (p==NULL)
        return false;
    LNode *q=p->next; //令q指向*p的后继结点
    p->data=p->next->data; //和后继结点交换数据域
    p->next=q->next; //将*q结点从链中“断开”
    free(q); //释放后继结点的存储空间
    return true;
}
  
```



我可真聪明

可能是指向一个结点，也可能是指向NULL



时间复杂度: $O(1)$

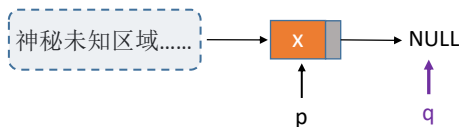
王道考研/CSKAOYAN.COM

22

指定结点的删除

```
//删除指定结点 p
bool DeleteNode (LNode *p){
    if (p==NULL)
        return false;
    LNode *q=p->next; //令q指向*p的后继结点
    p->data=p->next->data; //和后继结点交换数据域
    p->next=q->next; //将*q结点从链中“断开”
    free(q); //释放后继结点的存储空间
    return true;
}
```

单链表的局限性：
无法逆向检索，有
时候不太方便



如果p是最后
一个结点...

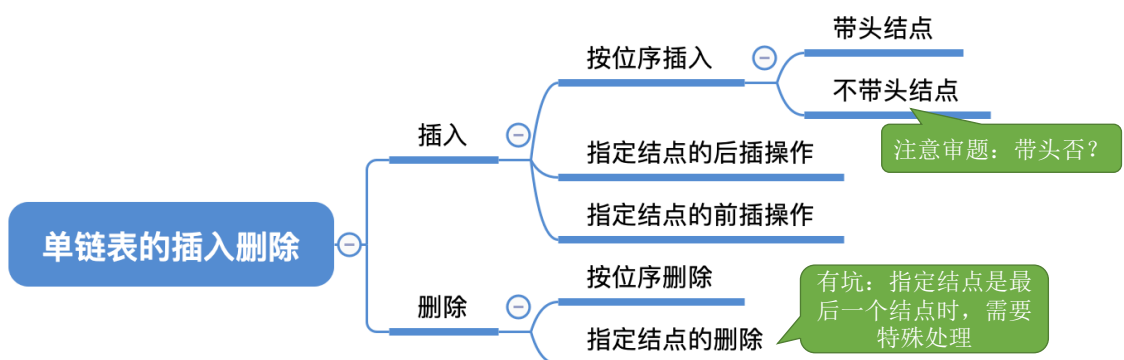
只能从表头开始依
次寻找p的前驱，
时间复杂度 $O(n)$



王道考研/CSKAOYAN.COM

23

知识回顾与重要考点



Tips:

1. 这些代码都要会写，都重要
2. 打牢基础，慢慢加速
3. 体会带头结点、不带头结点的代码区别
4. 体会“封装”的好处

王道考研/CSKAOYAN.COM

24

封装的好处

小功能模块化，代码逻辑清晰

```
//在第 i 个位置插入元素 e (带头结点)
bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
    LNode *p; //指针p指向当前扫描到的结点
    int j=0; //当前p指向的是第几个结点
    p = L; //L指向头结点，头结点是第0个结点 (不存数据)
    while (p!=NULL && j<i-1) { //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    if(p==NULL) //i值不合法
        return InsertNextNode(p, e);
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = e;
    s->next=p->next;
    p->next=s; //将结点s连到p之后
    return true; //插入成功
}
```

也可以称为“基本操作”。对书本的概念理解不要教条化

指针p指向第 i-1 个结点

p后插入新元素e

```
//后插操作：在p结点之后插入元素 e
bool InsertNextNode (LNode *p, ElemType e){
    if (p==NULL)
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    if (s==NULL) //内存分配失败
        return false;
    s->data = e; //用结点s保存数据元素e
    s->next=p->next;
    p->next=s; //将结点s连到p之后
    return true;
}
```

王道考研/CSKAOYAN.COM

25

知识回顾与重要考点

单链表的插入删除

插入

按位序插入

带头结点

不带头结点

指定结点的后插操作

指定结点的前插操作

删除

按位序删除

指定结点的删除

注意审题：带头否？

有坑：指定结点是最后一个结点时，需要特殊处理

Tips:

- 1. 这些代码都要会写，都重要
- 2. 打牢基础，慢慢加速
- 3. 体会带头结点、不带头结点的代码区别
- 4. 体会“封装”的好处

王道考研/CSKAOYAN.COM

26

本节内容

单链表 查找

王道考研/CSKAOYAN.COM

1

知识总览

单链表的查找

按位查找

按值查找

注：本节只探讨“带头结点”的情况

GetElem(L,i): **按位查找**操作。获取表L中第i个位置的元素的值。

LocateElem(L,e): **按值查找**操作。在表L中查找具有给定关键字值的元素。

王道考研/CSKAOYAN.COM

2

按位查找

//在第 i 个位置插入元素 e (带头结点)

```
bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
    LNode *p; //指针p指向当前扫描到的结点
    int j=0; //当前p指向的是第几个结点
    p = L; //L指向头结点, 头结点是第0个结点 (不存数据)
    while (p!=NULL && j<i-1) { //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    if(p==NULL) //i值不合法
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = e;
    s->next=p->next;
    p->next=s; //将结点s连到p之后
    return true; //插入成功
}
```

```
bool ListDelete(LinkList &L, int i, ElemType &e){
    if(i<1)
        return false;
    LNode *p; //指针p指向当前扫描到的结点
    int j=0; //当前p指向的是第几个结点
    p = L; //L指向头结点, 头结点是第0个结点 (不存数据)
    while (p!=NULL && j<i-1) { //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    if(p==NULL) //i值不合法
        return false;
    if(p->next == NULL) //第i-1个结点之后已无其他结点
        return false;
    LNode *q=p->next; //令q指向被删除结点
    e = q->data; //用e返回元素的值
    p->next=q->next; //将*q结点从链中“断开”
    free(q); //释放结点的存储空间
    return true; //删除成功
}
```

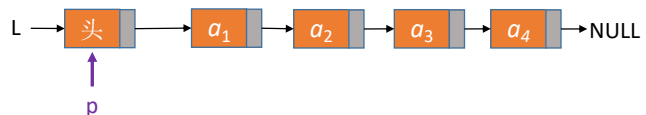
王道考研/CSKAOYAN.COM

3

按位查找

//按位查找, 返回第 i 个元素 (带头结点)

```
LNode * GetElem(LinkList L, int i){
    if(i<0)
        return NULL;
    LNode *p; //指针p指向当前扫描到的结点
    int j=0; //当前p指向的是第几个结点
    p = L; //L指向头结点, 头结点是第0个结点 (不存数据)
    while (p!=NULL && j<i) { //循环找到第 i 个结点
        p=p->next;
        j++;
    }
    return p;
}
```

边界情况:
①如果 $i=0$ 

王道考研/CSKAOYAN.COM

4

按位查找

//按位查找，返回第 i 个元素（带头结点）

```
LNode * GetElem(LinkList L, int i){
```

```
    if(i<0)
```

```
        return NULL;
```

```
    LNode *p;    //指针p指向当前扫描到的结点
```

```
    int j=0;    //当前p指向的是第几个结点
```

```
    p = L;    //L指向头结点，头结点是第0个结点（不存数据）
```

```
    while (p!=NULL && j<i) { //循环找到第  $i$  个结点
```

```
        p=p->next;
```

```
        j++;
```

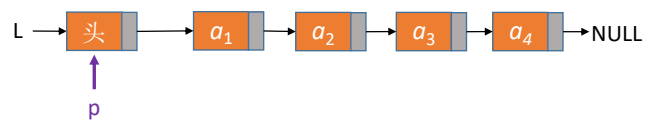
```
    }
```

```
    return p;
```

```
}
```

边界情况：

②如果 $i = 8$



王道考研/CSKAOYAN.COM

5

按位查找

//按位查找，返回第 i 个元素（带头结点）

```
LNode * GetElem(LinkList L, int i){
```

```
    if(i<0)
```

```
        return NULL;
```

```
    LNode *p;    //指针p指向当前扫描到的结点
```

```
    int j=0;    //当前p指向的是第几个结点
```

```
    p = L;    //L指向头结点，头结点是第0个结点（不存数据）
```

```
    while (p!=NULL && j<i) { //循环找到第  $i$  个结点
```

```
        p=p->next;
```

```
        j++;
```

```
    }
```

```
    return p;
```

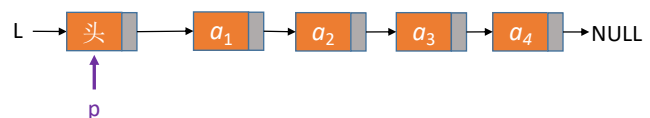
```
}
```

普通情况：

③如果 $i = 3$



脑补一下咯



平均时间复杂度： $O(n)$

王道考研/CSKAOYAN.COM

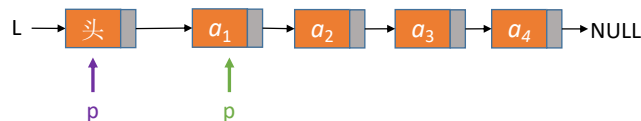
6

按位查找

王道书
版本

```
//按位查找, 返回第 i 个元素 (带头结点)
LNode * GetElem(LinkList L, int i){
    if(i<0)
        return NULL;
    LNode *p; //指针p指向当前扫描到的结点
    int j=0; //当前p指向的是第几个结点
    p = L; //L指向头结点, 头结点是第0个结点 (不存数据)
    while (p!=NULL && j<i) { //循环找到第 i 个结点
        p=p->next;
        j++;
    }
    return p;
}
```

```
LNode * GetElem(LinkList L, int i){
    int j=1;
    LNode *p=L->next;
    if(i==0)
        return L;
    if(i<1)
        return NULL;
    while(p!=NULL && j<i){
        p=p->next;
        j++;
    }
    return p;
}
```



王道考研/CSKAOYAN.COM

7

封装 (基本操作) 的好处

避免重复代码,
简洁、易维护

```
//在第 i 个位置插入元素 e (带头结点)
bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
    LNode *p = GetElem(L, i-1); //找到第i-1个结点
    p = L; //L指向头结点, 头结点是第0个结点 (不存数据)
    while (p!=NULL && j<i-1) { //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    if(p==NULL) //i值不合法
        return false;
    return InsertNextNode(p, e);
}

//插入操作: 在p结点之后插入元素 e
bool InsertNextNode (LNode *p, ElemType e){
    if (p==NULL)
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    if (s==NULL) //内存分配失败
        return false;
    s->data = e; //用结点s保存数据元素e
    s->next=p->next;
    p->next=s; //将结点s连到p之后
    return true;
}
```

```
bool ListDelete(LinkList &L, int i, ElemType &e){
    if(i<1)
        return false;
    LNode *p = GetElem(L, i-1); //找到第i-1个结点
    p = L; //L指向头结点, 头结点是第0个结点 (不存数据)
    while (p!=NULL && j<i-1) { //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    //后插操作: 在p结点之后插入元素 e
    bool InsertNextNode (LNode *p, ElemType e){
        if (p==NULL)
            return false;
        LNode *s = (LNode *)malloc(sizeof(LNode));
        if (s==NULL) //内存分配失败
            return false;
        s->data = e; //用结点s保存数据元素e
        s->next=p->next;
        p->next=s; //将结点s连到p之后
        return true;
    }
}
```

王道考研/CSKAOYAN.COM

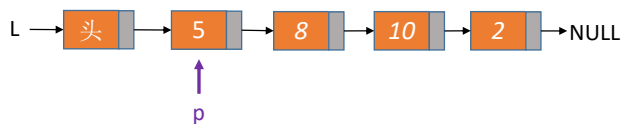
8

按值查找

```
//按值查找, 找到数据域==e 的结点
LNode * LocateElem(LinkList L, ElemType e) {
    ➡ LNode *p = L->next;
    //从第1个结点开始查找数据域为e的结点
    ➡ while (p != NULL && p->data != e)
    ➡     p = p->next;
    ➡ return p; //找到后返回该结点指针, 否则返回NULL
}
```

注: 假设本例中
ElemType 是 int

能找到的情况:
①如果 e = 8



王道考研/CSKAOYAN.COM

9

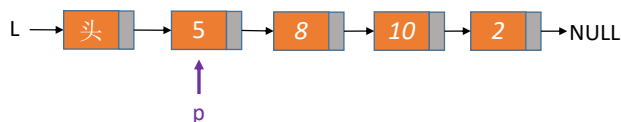
按值查找

```
//按值查找, 找到数据域==e 的结点
LNode * LocateElem(LinkList L, ElemType e) {
    ➡ LNode *p = L->next;
    //从第1个结点开始查找数据域为e的结点
    ➡ while (p != NULL && p->data != e)
    ➡     p = p->next;
    ➡ return p; //找到后返回该结点指针, 否则返回NULL
}
```

注: 假设本例中
ElemType 是 int

不能找到的情况:
②如果 e = 6

如果 ElemType 是更复杂
的结构类型呢?



平均时间复杂度: $O(n)$

王道考研/CSKAOYAN.COM

10

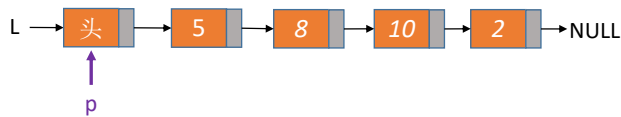
求表的长度

```
//求表的长度
int Length(LinkList L){
    int len = 0;    //统计表长
    LNode *p = L;
    while (p->next != NULL){
        p = p->next;
        len++;
    }
    return len;
}
```



如果不带头结点呢？

脑补一下咯

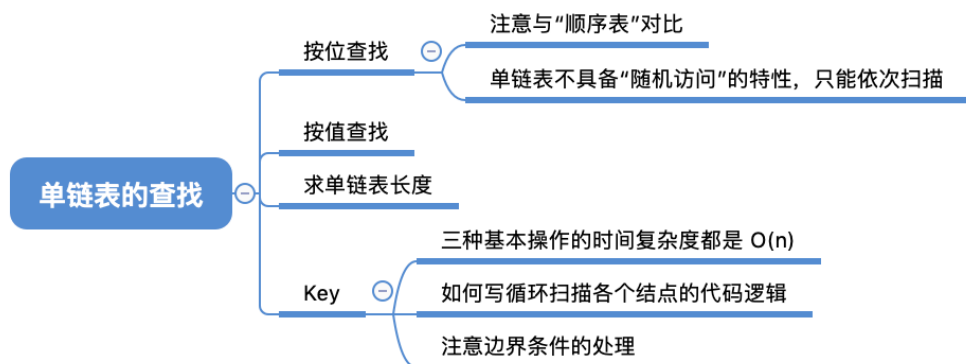


时间复杂度: $O(n)$

王道考研/CSKAOYAN.COM

11

知识回顾与重要考点



王道考研/CSKAOYAN.COM

12

本节内容

单链表 建立

王道考研/CSKAOYAN.COM

1

知识总览

单链表的建立

尾插法

头插法



如果给你很多个数据元素（ElemType），要把它们存到一个单链表里边，咋 neng 呢？

Step 1: 初始化一个单链表

Step 2: 每次娶一个数据元素，插入到表尾/表头

本节探讨带头
结点的情况

王道考研/CSKAOYAN.COM

2

尾插法建立单链表

```

typedef struct LNode{           //定义单链表结点类型
    ElemType data;              //每个节点存放一个数据元素
    struct LNode *next;         //指针指向下一个节点
}LNode, *LinkList;

//初始化一个单链表（带头结点）
bool InitList(LinkList &L) {
    L = (LNode *) malloc(sizeof(LNode)); //分配一个头结点
    if (L==NULL)                          //内存不足，分配失败
        return false;
    L->next = NULL;                        //头结点之后暂时还没有节点
    return true;
}

void test(){
    LinkList L; //声明一个指向单链表的指针
    //初始化一个空表
    InitList(L);
    //.....后续代码.....
}

```

L → 头 → NULL

王道考研/CSKAOYAN.COM

3

尾插法建立单链表

```

//在第 i 个位置插入元素 e（带头结点）
bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
    LNode *p; //指针p指向当前扫描到的结点
    int j=0;   //当前p指向的是第几个结点
    p = L;    //L指向头结点，头结点是第0个结点（不存数据）
    while (p!=NULL && j<i-1) { //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    if(p==NULL) //i值不合法
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = e;
    s->next=p->next;
    p->next=s; //将结点s连到p之后
    return true; //插入成功
}

```

尾插法建立单链表：

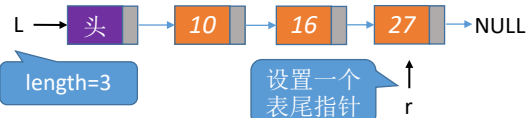
初始化单链表

设置变量 length 记录链表长度

While 循环 {

每次取一个数据元素 e;
ListInsert (L, length+1, e) 插到尾部;
length++;

每次都从头开始之后遍历，时间复杂度为 $O(n^2)$



王道考研/CSKAOYAN.COM

4

尾插法建立单链表

//后插操作: 在p结点之后插入元素 e

```
bool InsertNextNode (LNode *p, ElemType e){
    if (p==NULL)
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    if (s==NULL) //内存分配失败
        return false;
    s->data = e; //用结点s保存数据元素e
    s->next=p->next;
    p->next=s; //将结点s连到p之后
    return true;
}
```

后插操作



王道考研/CSKAOYAN.COM

5

尾插法建立单链表

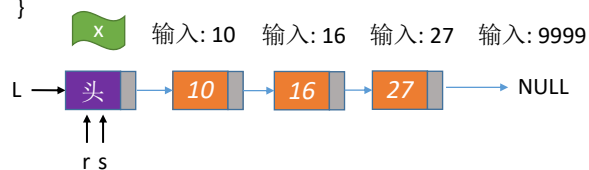
```
LinkedList List_TailInsert(LinkedList &L){ //正向建立单链表
    int x; //设ElemType为整型
    L=(LinkedList)malloc(sizeof(LNode)); //建立头结点
    LNode *s,*r=L; //r为表尾指针
    scanf("%d",&x); //输入结点的值
    while(x!=9999){ //输入9999表示结束
        s=(LNode *)malloc(sizeof(LNode));
        s->data=x;
        r->next=s;
        r=s; //r指向新的表尾结点
        scanf("%d",&x);
    }
    r->next=NULL; //尾结点指针置空
    return L;
}
```

初始化空表

在r结点之后插入元素 x

时间复杂度: $O(n)$

永远保持 r 指向最后一个结点



如果不带头结点呢?



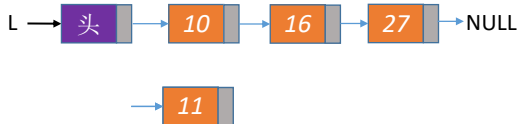
动口不动手

王道考研/CSKAOYAN.COM

6

头插法建立单链表

对头结点的
后插操作



```

//后插操作：在p结点之后插入元素 e
bool InsertNextNode (LNode *p, ElemType e){
    if (p==NULL)
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    if (s==NULL) //内存分配失败
        return false;
    s->data = e; //用结点s保存数据元素e
    s->next=p->next;
    p->next=s; //将结点s连到p之后
    return true;
}
  
```

头插法建立单链表：

初始化单链表

```

While 循环 {
    每次取一个数据元素 e;
    InsertNextNode (L, e);
}
  
```

王道考研/CSKAOYAN.COM

7

头插法建立单链表

```

LinkedList List_HeadInsert(LinkedList &L){ //逆向建立单链表
    LNode *s;
    int x;
    L=(LinkedList)malloc(sizeof(LNode)); //创建头结点
    L->next=NULL; //初始为空链表
    scanf("%d",&x); //输入结点的值
    while(x!=9999){ //输入9999表示结束
        s=(LNode*)malloc(sizeof(LNode)); //创建新结点
        s->data=x;
        s->next=L->next;
        L->next=s; //将新结点插入表中，L为头指针
        scanf("%d",&x);
    }
    return L;
}
  
```

重要应用!!!
链表的逆置



养成好习惯，只要是
初始化单链表，都先
把头指针指向 NULL

如果去掉这
一句呢？

```

//后插操作：在p结点之后插入元素 e
bool InsertNextNode (LNode *p, ElemType e){
    if (p==NULL)
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    if (s==NULL) //内存分配失败
        return false;
    s->data = e; //用结点s保存数据元素e
    s->next=p->next;
    p->next=s; //将结点s连到p之后
    return true;
}
  
```

如果不带头结点呢？

一定要动手鸭



各位，多说无益，动手
吧.....

王道考研/CSKAOYAN.COM

8

知识回顾与重要考点

头插法、尾插法：核心就是初始化操作、指定结点的后插操作

注意设置一个指向
表尾结点的指针



头插法的重要应用：链表的逆置

动手试一试：给你一个 LinkList
L，如何逆置？



王道考研/CSKAOYAN.COM

本节内容

双链表

王道考研/CSKAOYAN.COM

1

知识总览

双链表

初始化

插入

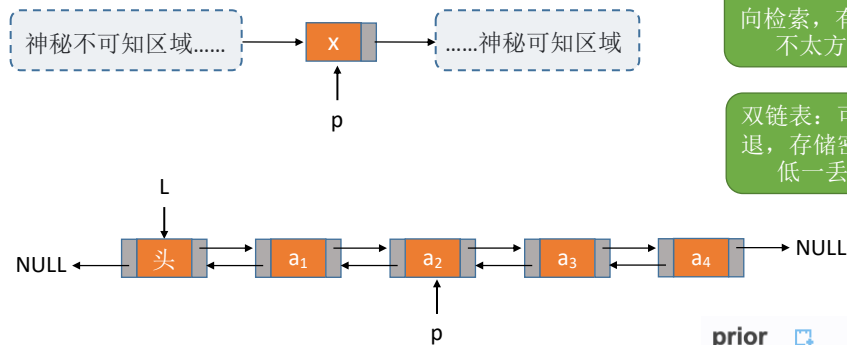
删除

遍历

王道考研/CSKAOYAN.COM

2

单链表 v.s. 双链表



单链表：无法逆向检索，有时候不太方便

双链表：可进可退，存储密度更低一丢丢



```
typedef struct DNode{
    ElemType data;
    struct DNode *prior, *next;
}DNode, *DLinklist;
```

//定义双链表结点类型
//数据域
//前驱和后继指针

prior

英 ['praɪə(r)] 美 ['praɪər]

adj. (时间、顺序等) 先前的; 优先的
n. 男修道院副院长; 托钵会会长; (非正式) 犯罪前科; 先验;
n. (Prior) (美) 普摩尔 (人名)
[复数 priors]

王道考研/CSKAOYAN.COM

3

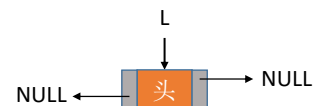
双链表的初始化（带头结点）

```
//初始化双链表
bool InitDLinklist(DLinklist &L){
    L = (DNode *) malloc(sizeof(DNode)); //分配一个头结点
    if (L==NULL) //内存不足，分配失败
        return false;
    L->prior = NULL; //头结点的 prior 永远指向 NULL
    L->next = NULL; //头结点之后暂时还没有节点
    return true;
}
```

```
void testDLinklist() {
    //初始化双链表
    DLinklist L;
    InitDLinklist(L);
    //后续代码。。。
}

//判断双链表是否为空（带头结点）
bool Empty(DLinklist L) {
    if (L->next == NULL)
        return true;
    else
        return false;
}
```

```
typedef struct DNode{
    ElemType data;
    struct DNode *prior, *next;
}DNode, *DLinklist;
```

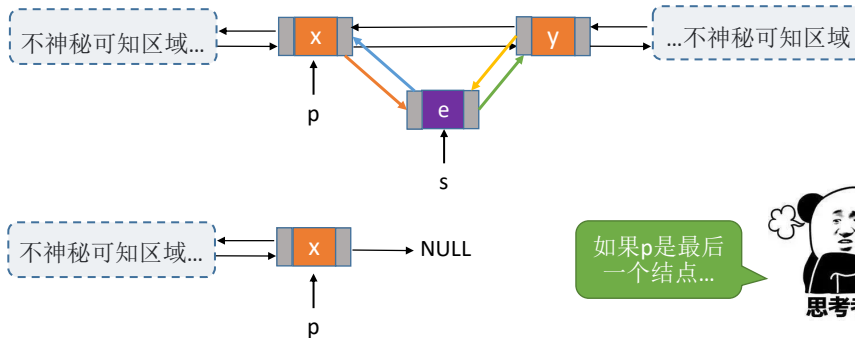
DLinklist $\longleftrightarrow$ 等价 $\longleftrightarrow$ DNode *

王道考研/CSKAOYAN.COM

4

双链表的插入

```
//在p结点之后插入s结点
bool InsertNextDNode(DNode *p, DNode *s){
    s->next=p->next;    //将结点*s插入到结点*p之后
    p->next->prior=s;
    s->prior=p;
    p->next=s;
}
```



王道考研/CSKAOYAN.COM

5

双链表的插入

```
//在p结点之后插入s结点
bool InsertNextDNode(DNode *p, DNode *s){
    if (p==NULL || s==NULL)    //非法参数
        return false;
    ① s->next=p->next;
    ② if (p->next != NULL)    //如果p结点有后继结点
        p->next->prior=s;
    ③ s->prior=p;
    ④ p->next=s;
    return true;
}
```

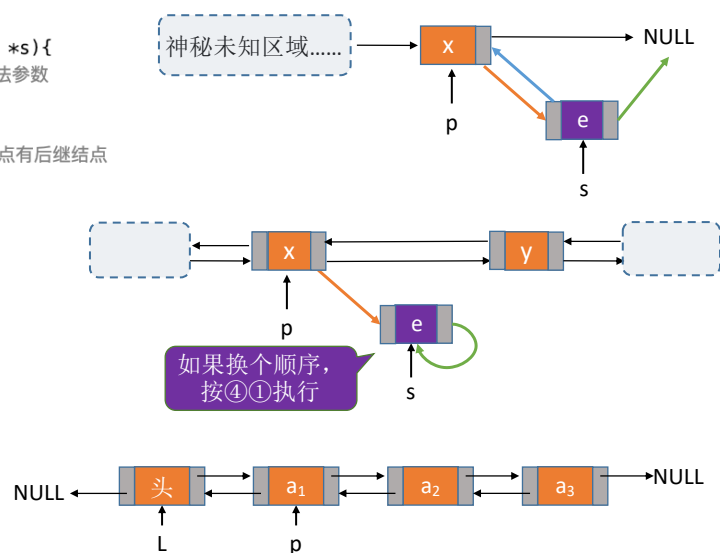
修改指针时要注意顺序

用后插操作实现结点的插入有什么好处?

按位序插入
前插操作



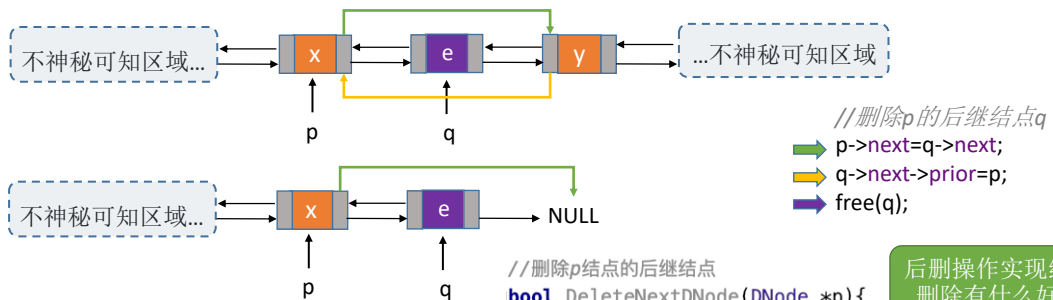
思考考



王道考研/CSKAOYAN.COM

6

双链表的删除



```
void DestoryList(DLinklist &L){
    //循环释放各个数据结点
    while (L->next != NULL)
        DeleteNextDNode(L);
    free(L); //释放头结点
    L=NULL; //头指针指向NULL
}
```

销毁表时
才能删除
头结点

```
//删除p结点的后继结点
bool DeleteNextDNode(DNode *p){
    if (p==NULL) return false;
    DNode *q = p->next; //找到p的后继结点q
    if (q==NULL) return false; //p没有后继
    p->next=q->next;
    if (q->next!=NULL) //q结点不是最后一个结点
        q->next->prior=p;
    free(q); //释放结点空间
    return true;
}
```

后删除操作实现结点的
删除有什么好处?

王道考研/CSKAOYAN.COM

7

双链表的遍历

后向遍历

```
while (p!=NULL){
    //对结点p做相应处理，如打印
    p = p->next;
}
```

前向遍历

```
while (p!=NULL){
    //对结点p做相应处理
    p = p->prior;
}
```

前向遍历（跳过头结点）

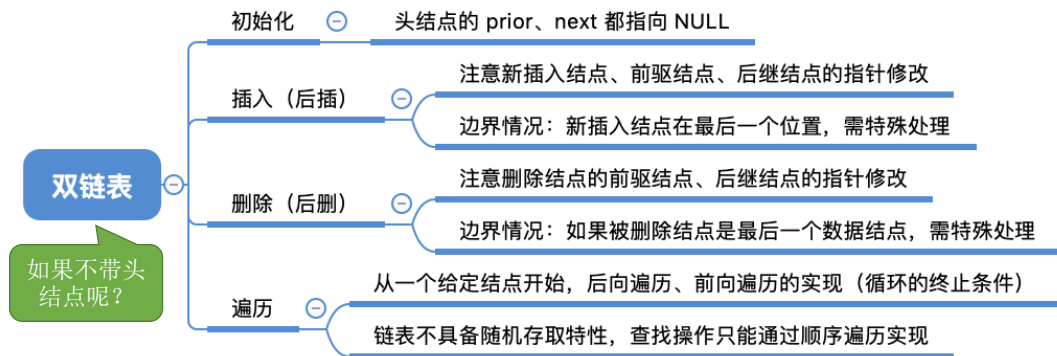
```
while (p->prior != NULL){
    //对结点p做相应处理
    p = p->prior;
}
```

双链表不可随机存取，按位查找、按值查找操作都只能用遍历的方式实现。时间复杂度 $O(n)$

王道考研/CSKAOYAN.COM

8

知识回顾与重要考点



王道考研/CSKAOYAN.COM

本节内容

循环链表

王道考研/CSKAOYAN.COM

1

知识总览

循环链表

循环单链表

循环双链表

就那么简单



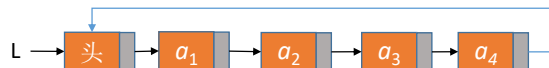
王道考研/CSKAOYAN.COM

2

循环单链表



单链表：表尾结点的next指针指向 NULL



循环单链表：表尾结点的next指针指向头结点

王道考研/CSKAOYAN.COM

3

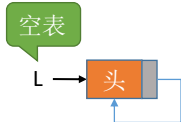
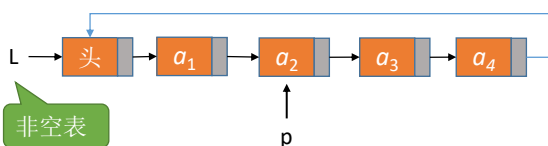
循环单链表

```
typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;
```

//定义单链表结点类型
//每个节点存放一个数据元素
//指针指向下一个节点

//初始化一个循环单链表

```
bool InitList(LinkList &L) {
    L = (LNode *) malloc(sizeof(LNode)); //分配一个头结点
    if (L==NULL) //内存不足，分配失败
        return false;
    L->next = L; //头结点next指向头结点
    return true;
}
```



//判断循环单链表是否为空

```
bool Empty(LinkList L) {
    if (L->next == L)
        return true;
    else
        return false;
}
```

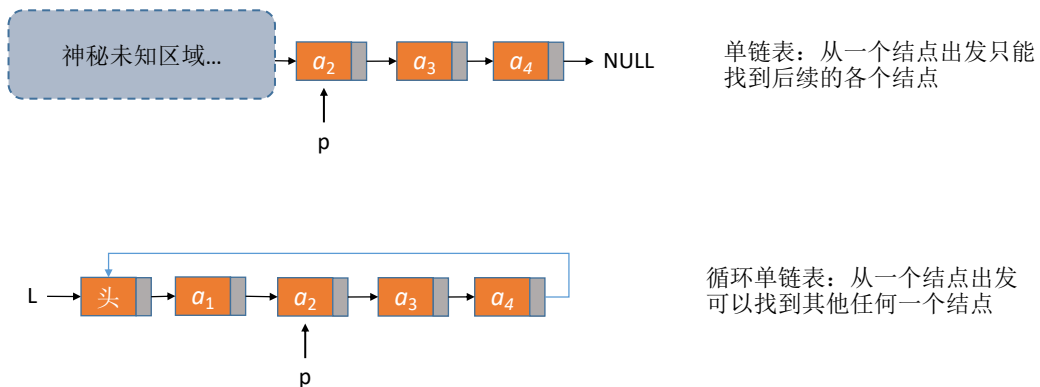
//判断结点p是否为循环单链表的表尾结点

```
bool isTail(LinkList L, LNode *p){
    if (p->next==L)
        return true;
    else
        return false;
}
```

王道考研/CSKAOYAN.COM

4

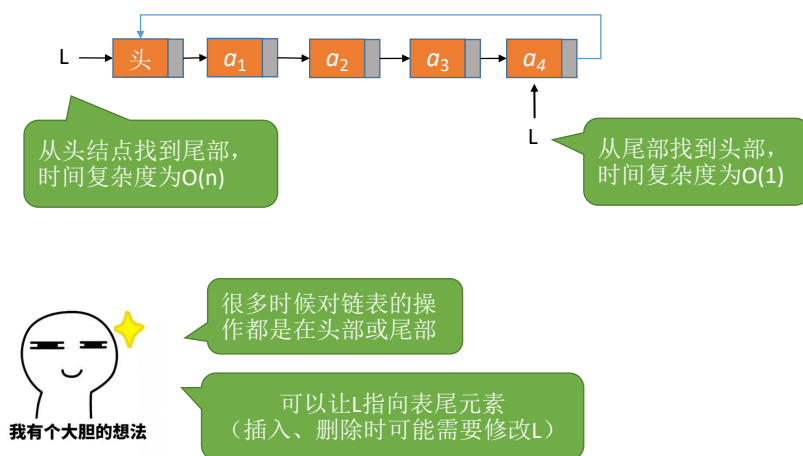
循环单链表



王道考研/CSKAOYAN.COM

5

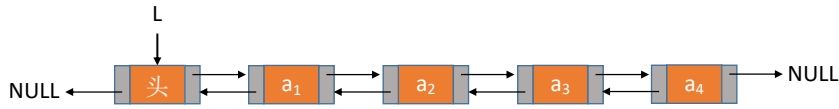
循环单链表



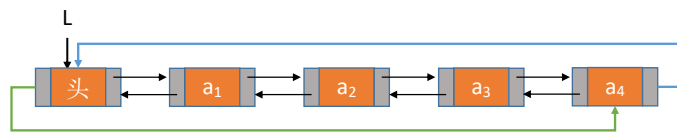
王道考研/CSKAOYAN.COM

6

循环双链表



双链表：
表头结点的 prior 指向 NULL；
表尾结点的 next 指向 NULL



循环双链表：
表头结点的 prior 指向表尾结点；
表尾结点的 next 指向头结点

王道考研/CSKAOYAN.COM

7

循环双链表的初始化

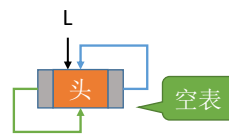
```
//初始化空的循环双链表
bool InitDLinkedList(DLinkedList &L){
    L = (DNode *) malloc(sizeof(DNode)); //分配一个头结点
    if (L==NULL) //内存不足, 分配失败
        return false;
    L->prior = L; //头结点的 prior 指向头结点
    L->next = L; //头结点的 next 指向头结点
    return true;
}

typedef struct DNode{
    ElemType data;
    struct DNode *prior,*next;
}DNode, *DLinkedList;

void testDLinkedList() {
    //初始化循环双链表
    DLinkedList L;
    InitDLinkedList(L);
    //...后续代码...
}

//判断循环双链表是否为空
bool Empty(DLinkedList L) {
    if (L->next == L)
        return true;
    else
        return false;
}

//判断结点p是否为循环单链表的表尾结点
bool isTail(DLinkedList L, DNode *p){
    if (p->next==L)
        return true;
    else
        return false;
}
```

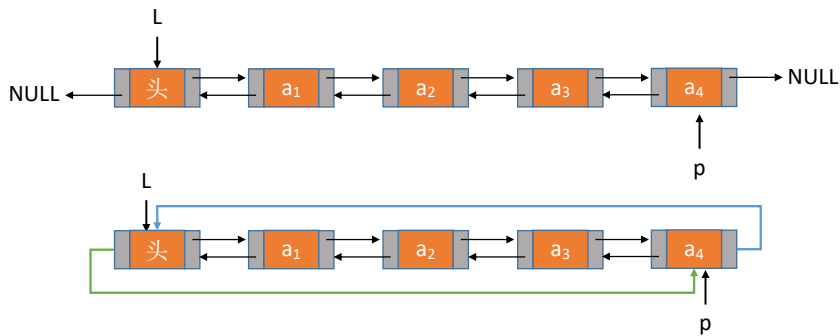


王道考研/CSKAOYAN.COM

8

双链表的插入

```
//在p结点之后插入s结点
bool InsertNextDNode(DNode *p, DNode *s){
    s->next=p->next;    //将结点*s插入到结点*p之后
    p->next->prior=s;
    s->prior=p;
    p->next=s;
}
```

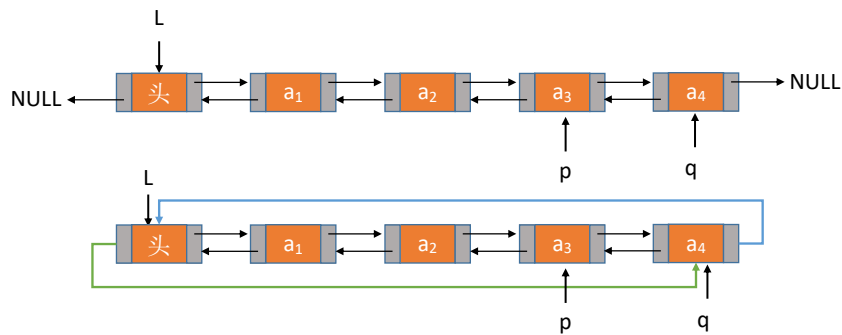


王道考研/CSKAOYAN.COM

9

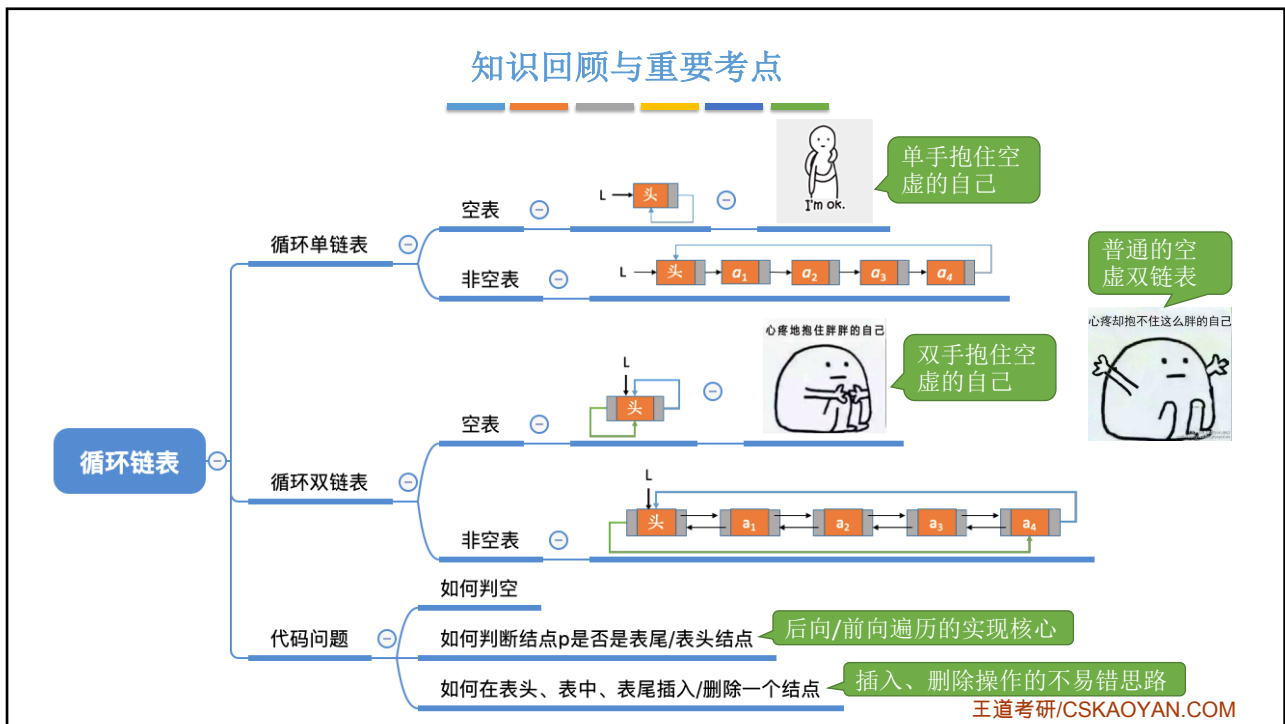
双链表的删除

```
//删除p的后继结点q
p->next=q->next;
q->next->prior=p;
free(q);
```



王道考研/CSKAOYAN.COM

10



本节内容

静态链表

王道考研/CSKAOYAN.COM

1

知识总览

静态链表

什么是静态链表

如何定义一个静态链表

简述基本操作的实现

王道考研/CSKAOYAN.COM

2

什么是静态链表

静态链表

addr	0	头	2
	1	e ₂	6
	2	e ₁	1
	3	e ₄	-1
	4		
	5		
	6	e ₃	3
	7		
	8		
	9		

数据元素

内存

0号结点充当“头结点”

游标为 -1 表示已经到达表尾

游标充当“指针”

下一个结点的数组下标 (游标)

单链表: 各个结点在内存中星罗棋布、散落天涯。

静态链表: 分配一整片连续的内存空间, 各个结点集中安置。

每个数据元素 4B, 每个游标4B (每个结点共 8B)
设起始地址为 **addr**

e₁ 的存放地址为 **addr + 8*2**

单链表

L → 头 → e₁ → e₂ → e₃ → e₄ → NULL

数据元素

指向下一个结点的指针 (地址)

王道考研/CSKAOYAN.COM

3

用代码定义一个静态链表

a

0		
1		
2		
3		
4		
5		
6		
7		
8		
9		

data (数据元素)

next (游标)

内存

```
#define MaxSize 10 //静态链表的最大长度
struct Node{ //静态链表结构类型的定义
    ElemType data; //存储数据元素
    int next; //下一个元素的数组下标
};

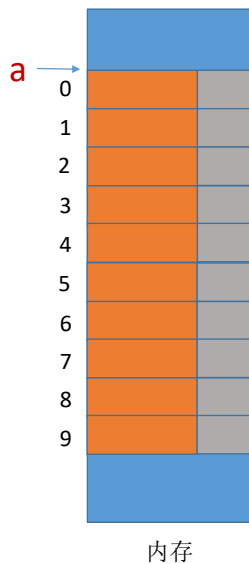
void testSLinkList() {
    struct Node a[MaxSize]; //.....后续代码
}
```

数组 a 作为静态链表

王道考研/CSKAOYAN.COM

4

用代码定义一个静态链表



```
#define MaxSize 10 //静态链表的最大长度
typedef struct {    //静态链表结构类型的定义
    ElemType data;  //存储数据元素
    int next;       //下一个元素的数组下标
} SLinkList[MaxSize];
```



```
#define MaxSize 10 //静态链表的最大长度
struct Node{        //静态链表结构类型的定义
    ElemType data;  //存储数据元素
    int next;       //下一个元素的数组下标
};
typedef struct Node SLinkList[MaxSize];
```

可用 SLinkList 定义 “一个长度为 MaxSize 的 Node 型数组”

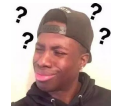
```
void testSLinkList() {
    SLinkList a;
    //.....后续代码
}
```

a 是一个静态链表



```
void testSLinkList() {
    struct Node a[MaxSize];
    //.....后续代码
}
```

a 是一个 Node 型数组



王道考研/CSKAOYAN.COM

5

对猜想的验证

```
#define MaxSize 10 //静态链表的最大长度
struct Node{        //静态链表结构类型的定义
    int data;        //存储数据元素
    int next;        //下一个元素的数组下标
};
typedef struct {    //静态链表结构类型的定义
    int data;        //存储数据元素
    int next;        //下一个元素的数组下标
} SLinkList[MaxSize];

void testSLinkList() {
    struct Node x;
    printf("sizeX=%d\n", sizeof(x));

    struct Node a[MaxSize];
    printf("sizeA=%d\n", sizeof(a));

    SLinkList b;
    printf("sizeB=%d\n", sizeof(b));
}
```

结论:

SLinkList b —— 相当于定义了一个长度为 MaxSize 的 Node 型数组

运行结果

```
sizeX=8
sizeA=80
sizeB=80
```

Process finished with exit code 0

王道考研/CSKAOYAN.COM

6

简述基本操作的实现

静态链表

a	0	头	-1
	1		
	2		
	3		
	4		
	5		
	6		
	7		
	8		
	9		

数据元素

内存

```
#define MaxSize 10
typedef struct {
    ElemType data;
    int next;
} SLinkList[MaxSize];
```

初始化静态链表:

把 a[0] 的 next 设为 -1

把其他结点的 next 设为一个特殊值用来表示结点空闲, 如 -2

```
void testSLinkList() {
    SLinkList a;
    //.....后续代码
}
```

单链表

L → 头 → NULL

王道考研/CSKAOYAN.COM

7

简述基本操作的实现

静态链表

a	0	头	2
	1	e ₂	6
	2	e ₁	1
	3	e ₄	-1
	4		
	5		
	6	e ₃	3
	7		
	8		
	9		

数据元素

内存

```
#define MaxSize 10
typedef struct {
    ElemType data;
    int next;
} SLinkList[MaxSize];
```

查找:

从头结点出发挨个往后遍历结点

插入位序为 i 的结点:

① 找到一个空的结点, 存入数据元素
② 从头结点出发找到位序为 i-1 的结点
③ 修改新结点的 next
④ 修改 i-1 号结点的 next

删除某个结点:

① 从头结点出发找到前驱结点
② 修改前驱结点的游标
③ 被删除结点 next 设为 -2

```
void testSLinkList() {
    SLinkList a;
    //.....后续代码
}
```

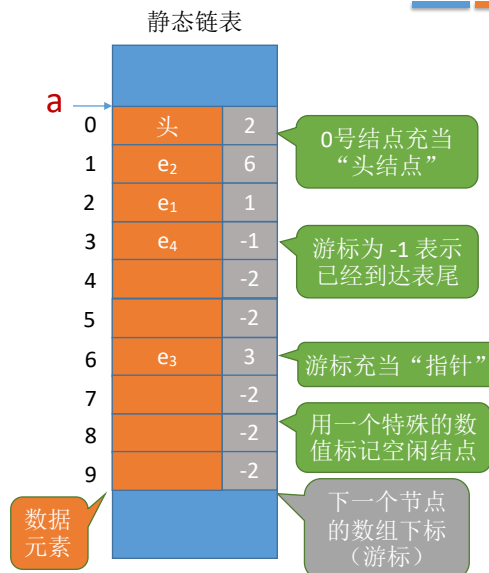
如何让 next 为某个特殊值, 如 -2

如何判断结点是否为空?

王道考研/CSKAOYAN.COM

8

知识回顾与重要考点



```

#define MaxSize 10 //静态链表的最大长度
typedef struct {    //静态链表结构类型的定义
    ElemType data;  //存储数据元素
    int next;       //下一个元素的数组下标
} SLinkList[MaxSize];

void testSLinkList() {
    SLinkList a;
    //.....后续代码
}

```

静态链表：用数组的方式实现的链表

优点：增、删 操作不需要大量移动元素

缺点：不能随机存取，只能从头结点开始依次往后查找；容量固定不可变

适用场景：①不支持指针的低级语言；②数据元素数量固定不变的场景（如操作系统的文件分配表FAT）

王道考研/CSKAOYAN.COM

本节内容

顺序表
V.S.
链表

王道考研/CSKAOYAN.COM

1

知识总览

顺序表
V.S.
链表

Round 1

逻辑结构

Round 2

物理结构/存储结构

Round 3

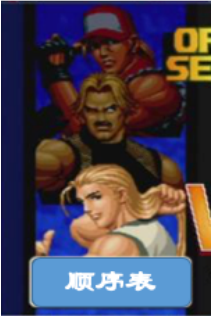
数据的运算/基本操作

如何抉择? ❤️到底爱谁? 💜

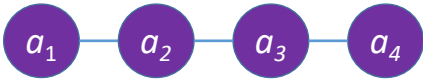
王道考研/CSKAOYAN.COM

2

Round 1: 逻辑结构



顺序表



a_1 — a_2 — a_3 — a_4



链表

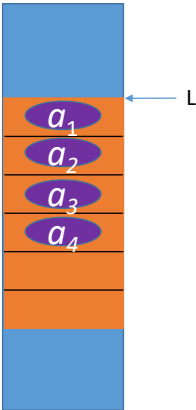
都属于线性表，都是线性结构

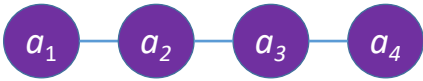
王道考研/CSKAOYAN.COM

3

Round 2: 存储结构

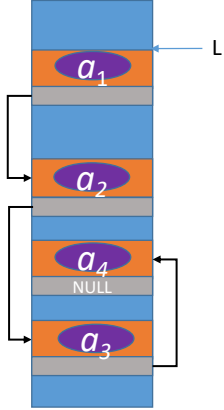
顺序表
(顺序存储)





a_1 — a_2 — a_3 — a_4

链表
(链式存储)



都属于线性表，都是线性结构

优点：支持随机存取、存储密度高
缺点：大片连续空间分配不方便，改变容量不方便

优点：离散的小空间分配方便，改变容量方便
缺点：不可随机存取，存储密度低

王道考研/CSKAOYAN.COM

4

Round 3: 基本操作



复习回忆思路:

创、增删改查

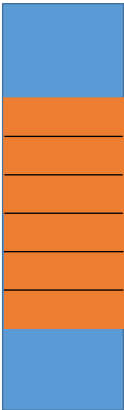
王道考研/CSKAOYAN.COM

5

Round 3: 基本操作

创

顺序表
(顺序存储)



需要预分配大片连续空间。若分配空间过小，则之后不方便拓展容量；若分配空间过大，则浪费内存资源

静态分配: 静态数组
动态分配: 动态数组 (malloc、free)

容量不可改变

容量可改变，但需要移动大量元素，时间代价高

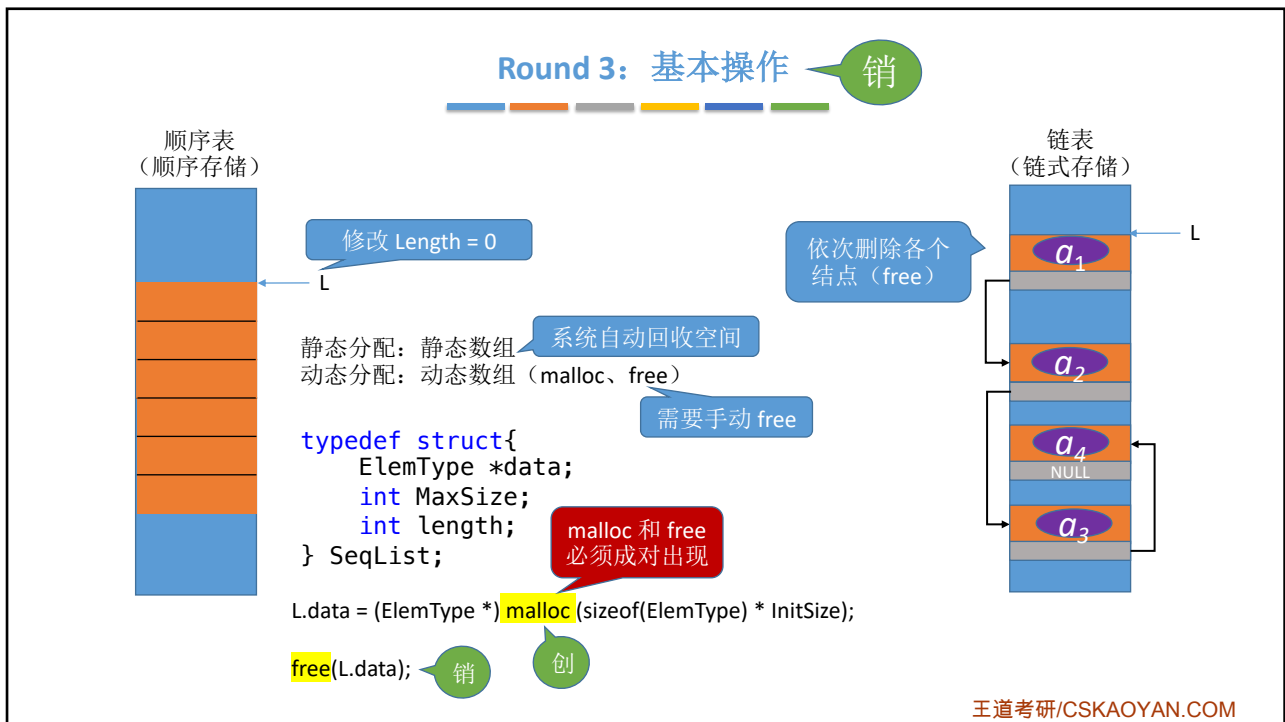
链表
(链式存储)



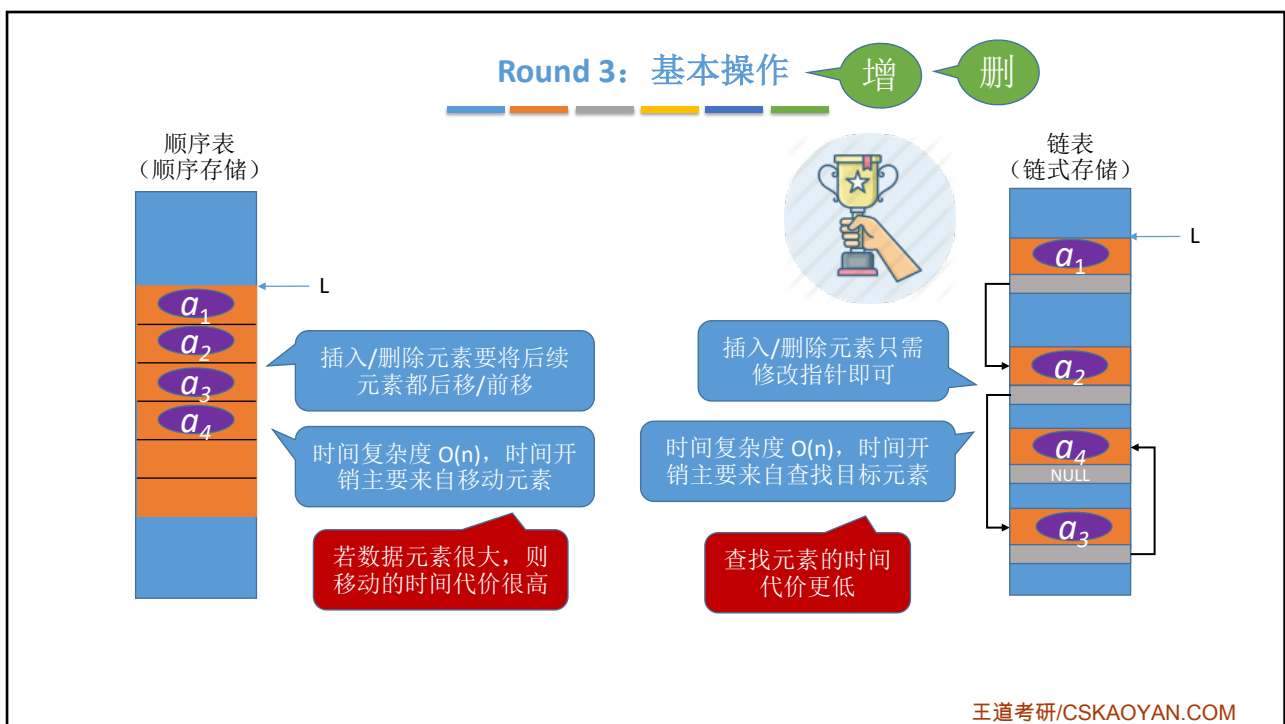
只需分配一个头结点 (也可以不要头结点，只声明一个头指针)，之后方便拓展

王道考研/CSKAOYAN.COM

6

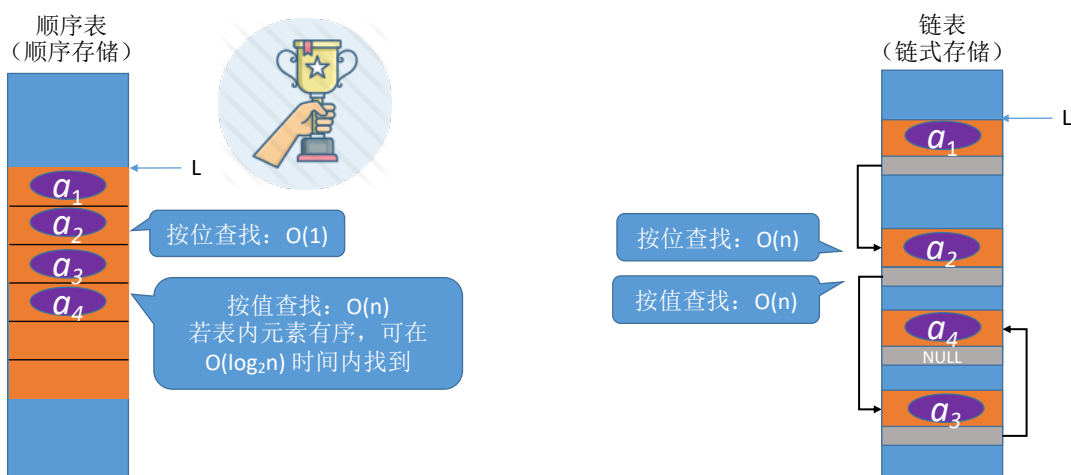


7



8

Round 3: 基本操作 查



王道考研/CSKAOYAN.COM

9

用顺序表 or 链表?

	顺序表	链表
弹性（可扩容）	😭	😄
增、删	😭	😄
查	😄	😭

表长难以预估、经常要增加/删除元素

——链表

表长可预估、查询（搜索）操作较多

——顺序表



王道考研/CSKAOYAN.COM

10

知识回顾与重要考点



开放式问题的答题思路:

问题： 请描述顺序表和链表的 bla bla bla...
实现线性表时，用顺序表还是链表好？

顺序表和链表的**逻辑结构**都是线性结构，都属于线性表。

但是二者的**存储结构**不同，顺序表采用顺序存储...(特点，带来的优点缺点)；链表采用链式存储...(特点、导致的优缺点)。

由于采用不同的存储方式实现，因此**基本操作**的实现效率也不同。当初始化时...；当插入一个数据元素时...；当删除一个数据元素时...；当查找一个数据元素时...

王道考研/CSKAOYAN.COM